PROJECT DOCUMENTATION & INTERVIEW HANDBOOK

Repositories VikramMali14/HueVista · VikramMali14/HueVistaFrontEnd

Backend Spring Boot 4.1 · Java 17 · PostgreSQL · Redis · AWS S3

Frontend Next.js 16 (App Router) · React 19 · TypeScript · WebGL2

Payments Razorpay Subscriptions + Orders + HMAC webhooks

AI Claude (Haiku Vision / Sonnet) · Replicate (FLUX.2, Nano Banana Pro, SAM 2)

Scale ~78,000 LOC · 207 REST endpoints · 43 entities · 479 automated tests

Contents

An AI-powered paint-shade visualiser and B2B SaaS platform for the Indian paint retail trade

1 · The 60-second pitch

> This is the answer to *"tell me about your project"*. Learn the shape of it, not the words.

THE PITCH

"HueVista is a B2B SaaS platform I built for the Indian paint retail trade. A paint shop's walk-in customer uploads a photo of their room, and the app repaints the walls in real catalogue shades — Asian Paints, Berger, Nerolac, Dulux, Nippon — photorealistically, before a single stroke is painted.

The interesting engineering is in three places. **First**, the segmentation pipeline: I don't ask a generative model to redraw the room, because that costs ₹3–8 per render, takes 10–30 seconds, and hallucinates the customer's furniture. Instead I use AI *once* to produce a colour-coded mask of the paintable surfaces, split that server-side into per-surface binary masks, and then do the actual recolouring in a WebGL2 fragment shader in the browser — 60 fps, zero marginal cost, and it's literally the original photo with the hue replaced, so it can't hallucinate.

Second, the money: full Razorpay integration with Subscriptions for monthly plans, Orders for one-off purchases, HMAC-SHA256 webhook verification that fails closed, idempotent event processing, and an atomic reserve-then-commit quota system so a failed AI run refunds the customer's credit.

Third, the security architecture: the browser never holds a JWT. Next.js middleware refreshes tokens into HttpOnly cookies and a BFF proxy route attaches the Authorization header server-side, so an XSS bug can't exfiltrate a session.

It's about 78,000 lines across two repositories, 207 REST endpoints, 43 JPA entities and 479 automated tests."

1.1 · The numbers you should know cold

78k	207	43
LINES OF CODE	REST ENDPOINTS	JPA ENTITIES

479	37	~8,000
AUTOMATED TESTS	FLYWAY MIGRATIONS	PAINT SHADES SEEDED

Metric	Value
Backend Java files / LOC	376 files · 33,435 lines
Backend test files / LOC / cases	53 files · 12,107 lines · 439 `@Test` methods
Frontend TS/TSX files / LOC	258 files · 44,913 lines
Frontend test files	40 Vitest suites
Next.js routes (pages)	38
Spring `@RestController` classes	34
Feature modules (backend packages)	17 (auth, image, paint, project, ai, billing, account, hierarchy, admin, store, painter, lead, support, newsletter, notification, guest, common)
External services integrated	Razorpay · Anthropic Claude · Replicate · AWS S3 · Google OAuth2 · SMTP · Twilio SMS

2 · The problem, the market and the business model

2.1 · The problem

Choosing a wall colour is the slowest, most anxious step in a paint purchase. The customer stands in the shop holding a 2 cm × 3 cm paper swatch from a fan deck and is asked to imagine it across 400 square feet of their living room, under their lighting, next to their furniture. They can't. So they hesitate, take the fan deck home, come back next week — or never.

For the retailer this is a **conversion problem**: a long, indecisive colour conversation blocks the counter and often ends in no sale. Sample pots and physical trials cost money and days.

HueVista compresses that conversation. The shopkeeper photographs the customer's room on a phone, the app repaints it in real catalogue shades in seconds, and the customer sees *their own room* in the colour — not a stock render.

2.2 · Why B2B and not B2C

A consumer app would need to acquire every homeowner individually — expensive, and each user paints once every seven years. A paint retailer, by contrast, has a colour conversation twenty times a day, already owns the customer relationship, and will happily pay a monthly fee for a tool that closes sales faster. So the product is sold to shops, and shops hand it to walk-ins.

Distribution follows the trade hierarchy that already exists in the Indian paint market:

```
Manufacturer  ->  Distributor  ->  Retailer (shop)  ->  Painter  ->  End customer
                       |                |                  |             |
                  creates shops    issues access      works under    redeems a code,
                  assigns brands   codes to walk-ins   the shop      gets N days of access
                  assigns features
```

That hierarchy is modelled directly in code as five roles — `ADMIN`, `DISTRIBUTOR`, `RETAILER`, `PAINTER`, `CUSTOMER` — plus an anonymous `GUEST` principal for walk-ins who never create an account.

2.3 · The business model — three revenue rails

Rail	Razorpay product	How it works
Monthly plan	Subscriptions	A shop subscribes to Starter / Professional / Business. Each tier carries a monthly quota of *projects*.
Points	Orders (one-time)	₹1 = 1 point. Points are a closed-loop currency spent on overage — extra projects, reopens. They expire after 365 days.
In-store kiosk	Orders (one-time)	A walk-in scans the shop's public link and pays a flat ₹99 for one visualisation. The shop earns 30 reward points; it takes no cash share.

The pricing ladder (source of truth: `Plan.java`)

Tier	Price / month	Projects / cycle	PDF images per board	PDF downloads / cycle	Extra project (points)	Extra project (cash)
FREE (trial)	₹0	2	4	5	80	₹99
Starter	₹999	15	4	25	60	₹65
Professional	₹2,499	45	8	100	50	₹55
Business	₹4,999	100	12	300	40	₹45
Enterprise	Custom	Unlimited	16	Unlimited	40 (quoted only)	₹45 (quoted only)

> **DESIGN DECISION WORTH EXPLAINING IN AN INTERVIEW**
>
> **The billable unit is a *project*, not an API call.** An earlier design charged separately for "an extra image" and "an extra auto-mask" — which meant a shop could buy the wrong one and still be unable to finish a run: a cleaned photo with no way to mask it. One unit, one purchase, one price. The pipeline either completes or it refunds.
>
> **The extra-project rate gets cheaper as the tier gets bigger** (80 → 60 → 50 → 40 points). A shop that keeps outgrowing its plan is nudged *up a tier* rather than penalised for staying. An account with no paid plan pays the FREE rate — the dearest — so subscribing is always the cheaper way to buy volume.

Unit economics — why the architecture is what it is

Pipeline step	Provider	Approx cost / run	Quota-charged?
Upload classification	Claude Haiku Vision	~₹0.30	No — per-IP rate-limited instead
Photo clean-up (opt-in)	Replicate · Nano Banana Pro	~₹8.50	Yes (part of the project)
Auto colour-coded mask	Replicate · FLUX.2 [max]	~₹3.40	Yes (part of the project)
Click-to-segment refinement	Replicate · SAM 2	~₹1.70	No — free and unlimited
Colour recommendation	Claude Sonnet Vision	~₹2.50	Yes
Applying a colour to a wall	**Browser WebGL2**	**₹0**	**Unlimited, forever**

That last row is the whole architectural thesis. The expensive AI work happens **once per project**. Everything the customer actually does at the counter — trying twelve shades, comparing, undoing — is free, instant and local.

3 · Complete technology stack

3.1 · Backend

Concern	Technology	Why / notes
Language	Java 17	Records, sealed types, pattern-matching `switch`, text blocks used throughout.
Framework	Spring Boot 4.1.0	Spring Framework 7 / Spring Security 7. Boot 4 splits auto-configuration per technology module — e.g. `spring-boot-flyway` is required; `flyway-core` alone is not auto-wired.
Build	Maven (with wrapper `./mvnw`)	AWS SDK BOM imported so all AWS module versions stay in lockstep.
Web layer	`spring-boot-starter-webmvc`	Blocking servlet stack on Tomcat. Deliberate: the workload is DB + HTTP-to-AI, and the AI calls are pushed off the request thread anyway.
Security	Spring Security 7 + JJWT 0.13.0	Stateless, HS256 JWT, custom filters, method security enabled.
OAuth2	`spring-boot-starter-oauth2-client`	Google sign-in with a custom success handler + one-time exchange code.
Persistence	Spring Data JPA / Hibernate 7	43 entities, Specifications API for the filterable shade catalogue.
Database	PostgreSQL 16 (prod) · H2 (dev + test)	H2 is `runtime` scope, not just test, so the `dev` profile boots with zero external infra.
Migrations	Flyway (37 versioned scripts)	Flyway owns the schema; Hibernate runs `ddl-auto=validate` and never mutates it.
Cache + queue + limiter	Redis (Spring Data Redis + Spring Cache)	Three jobs: shade-catalogue cache, reliable segmentation job queue, per-IP fixed-window rate limiter.
Object storage	AWS S3 SDK v2 (ap-south-1) → CloudFront	Pluggable behind a `StorageService` interface; falls back to local filesystem when no bucket is configured.
Payments	Razorpay Java SDK 1.4.9	Subscriptions, Orders, HMAC webhook verification.
AI	Anthropic Messages API · Replicate REST	Single `ClaudeService` client; Replicate polled with backoff.
Image processing	Thumbnailator 0.4.21 · Java2D / ImageIO	Resize-before-Claude cuts vision input tokens ~10×; ImageIO does the mask pixel work.
Mail / SMS	`spring-boot-starter-mail` · Twilio	Verification codes, password resets, billing receipts, admin 2FA.
API docs	springdoc-openapi 3.0.3 (Swagger UI)	Default OFF in production — the spec enumerates the whole attack surface.

Observability	Spring Actuator + Micrometer Prometheus	Only `/actuator/health` exposed by default; Prometheus gated behind two flags.
Boilerplate	Lombok	`@RequiredArgsConstructor`, `@Builder`, `@Slf4j`, `@Getter`.
Testing	JUnit 5 · Spring Boot Test · MockMvc · Mockito · H2	439 test methods; every external service mocked.

3.2 · Frontend

Concern	Technology	Why / notes
Framework	Next.js 16 (App Router)	React Server Components, streaming SSR, server actions, edge middleware.
UI runtime	React 19.2	Suspense boundaries around every app page.
Language	TypeScript 6 (strict)	879-line shared `types.ts` mirrors the backend DTO contract.
Styling	Plain CSS with custom-property design tokens	No Tailwind, no CSS-in-JS. "Midnight Spectrum" token system; light + dark themes.
Graphics	WebGL2 (hand-written GLSL) · Canvas 2D fallback	536-line recolour engine + a 351-line 2D fallback for devices without WebGL2.
Animation	GSAP · Motion · OGL	Marketing-page motion and the circular gallery.
Auth transport	HttpOnly cookies + BFF proxy route	No token ever reaches client JavaScript.
PDF export	Custom client-side generator (`pdf-export.ts`, 508 lines)	Colour boards produced in the browser from recoloured canvases.
Payments UI	Razorpay Checkout.js	Loaded lazily; CSP whitelists exactly the Razorpay hosts.
Testing	Vitest · Testing Library · jsdom	40 suites covering components, hooks and route guards.
Lint	ESLint 9 (flat config) + `eslint-config-next`	`npm run lint` and `npm run typecheck` in CI.

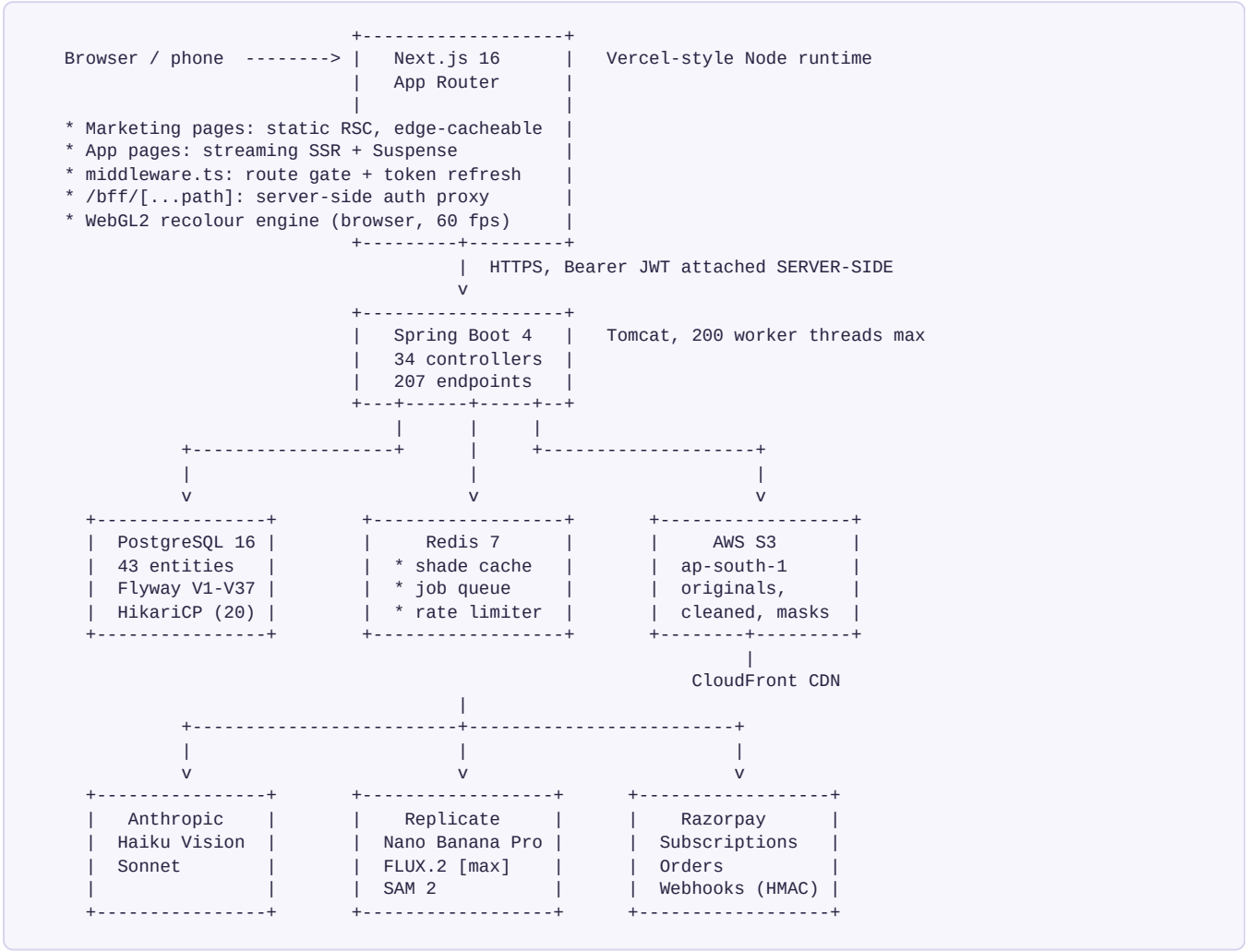
3.3 · AI / ML services

Task	Model	Behaviour
Image classification	Claude Haiku 4.5 (vision)	Returns exactly one word: `INDOOR`, `OUTDOOR` or `INVALID`. Image downsized to 1024 px / JPEG q85 first — ~10× fewer input tokens, same accuracy. Fails **hard** (the upload is rejected).
Photo clean-up (opt-in)	Replicate · Nano Banana Pro	Removes wires, clutter and bushes *and* repaints painted surfaces to a fresh near-white reference. The cleaned photo becomes the canvas, so masks align to it.
Auto-segmentation	Replicate · FLUX.2 [max] (image edit)	Edits the photo into flat colour blocks: **RED** = main wall, **GREEN** = accent wall, **BLUE** = trim, BLACK = everything else.
Click-to-segment	Replicate · Meta SAM 2	Single positive point prompt → mask of the clicked surface. The manual safety net; free and unlimited.
Colour recommendation	Claude Sonnet (vision)	Reads the room photo, proposes coordinated palettes, which are then snapped to real catalogue shades by ΔE matching.
Shade enrichment	Claude Sonnet	At catalogue seed time: style tags, mood descriptors, finish recommendations per shade.
Support agent	Claude Sonnet	In-app support widget. Fails **soft** — returns `Optional.empty()` and the UI degrades to a contact form.

> **COST-CONTROL FEATURE WORTH MENTIONING**
>
> `HUEVISTA_TESTING_STUB_AI_ENABLED=true` replaces the two paid Replicate calls with local no-ops — the mask is drawn locally as three equal vertical stripes (RED | GREEN | BLUE) so every downstream feature (region editing, shade picking, compare, PDF, share) still has real data to work with. Billing is deliberately *untouched*: a stub run charges credits exactly like a real one, so plan limits stay testable. This let the whole product be click-tested end-to-end for free.

4 · System architecture

4.1 · The whole system on one page



4.2 · Backend package structure — feature-sliced, not layer-sliced

The backend is organised by **business capability**, not by technical layer. There is no top-level `controllers/`, `services/`, `repositories/`. Each feature owns its full vertical slice:

```
com.gridstore.huevista
├── auth/          JWT, refresh tokens, Google OAuth2, OTP, admin 2FA, guest tokens
│   ├── config/    SecurityConfig, PasswordConfig
│   ├── controller/ dto/ filter/ handler/ model/ repository/ service/ util/
├── image/         upload, Claude Vision classification, S3 / local storage
├── paint/         Brand, PaintLine, Shade, catalogue search, seeders, colour maths
├── project/       Project + Region lifecycle, segmentation, mask processing, queue
├── ai/            colour recommendations, CIELAB ΔE matcher
├── billing/       Razorpay subscriptions + orders, points ledger, quotas, webhooks
├── account/       Organization, membership, access codes, entitlements, feature guards
├── hierarchy/     distributor → retailer → painter network, reporting
├── store/         public in-store kiosk links, wallet
├── painter/       painter profiles, invitations, paint jobs
├── lead/          public "request a shop account" flow + auto-approval job
├── support/       AI support agent, conversations, channel webhooks
├── newsletter/    subscribe / unsubscribe with token auth
├── notification/  EmailSender, SmsSender abstractions
├── guest/         guest-scoped endpoints for walk-ins with no account
├── admin/         admin console endpoints, data reset
└── common/        ai/ audit/ config/ exception/ ratelimit/ web/
```

WHY THIS MATTERS IN AN INTERVIEW

Feature-slicing means a change to billing touches one directory. It also makes the module boundaries the natural seams for extracting services later — `project/` and its Replicate calls are the obvious first candidate to split out, because it is the only CPU-and-latency-heavy module.

4.3 · The request path, end to end

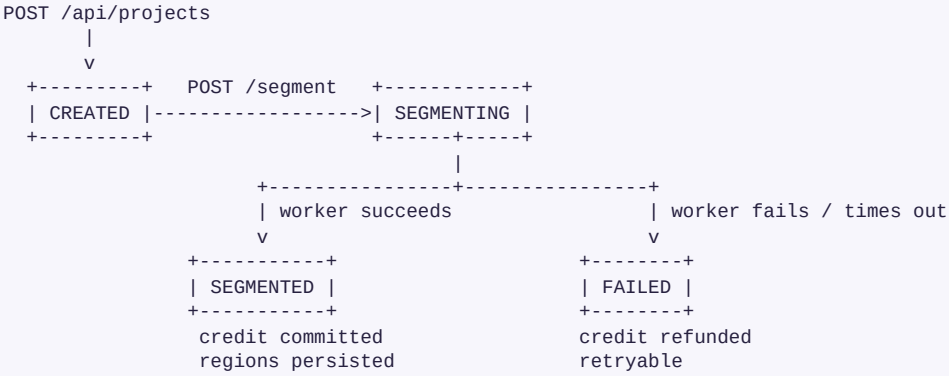
```
Browser fetch("/bff/api/projects")
    |
    v
[1] Next.js middleware.ts
      - is the path protected?         -> yes
      - hv_access cookie present?      -> no, expired
      - POST /api/auth/refresh with hv_refresh
      - write new hv_access cookie (middleware CAN mutate cookies)
    |
    v
[2] Next.js route handler /bff/[...path]/route.ts
      - isBffPathAllowed("api/projects")?   -> allow-list check, else 403
      - read hv_access from the HttpOnly cookie jar (server-side)
      - set Authorization: Bearer
      - set X-Forwarded-For: real client IP (so backend limiters bucket correctly)
      - fetch(internalApiOrigin + "/api/projects")
    |
    v
[3] Spring Security filter chain
      CorsFilter -> CsrfFilter(disabled) -> GuestAuthFilter -> JwtAuthFilter
      -> SensitiveEndpointRateLimitFilter -> AuthorizationFilter
    |
    v
[4] FeatureGuardInterceptor    (@RequiresFeature — is this page granted to the shop?)
    |
    v
[5] @RestController -> @Service -> @Repository -> PostgreSQL
    |
    v
[6] GlobalExceptionHandler maps any thrown exception to a typed JSON error
    |
    v
Response streams back through the BFF unchanged (only safe headers passed through)
```

5 · End-to-end user journey

5.1 · The walk-in customer flow (the money path)

```
1. Shop staff signs in                 JWT issued, refresh cookie set
       |
2. Staff generates an access code      POST /api/organizations/{id}/access-codes
       |                               8-char code, 3 / 7 / 14-day validity
       |                               A project CREDIT is held against it
       v
3. Customer redeems the code           POST /api/access-codes/redeem-account
       |                               -> auto-creates a passwordless CUSTOMER
       |                                  account and returns a full session
       |                               (or /redeem-guest -> anonymous guest JWT)
       v
4. Customer uploads a room photo       POST /api/images/upload
       |                               - MIME + magic-byte validation
       |                               - resize to 1024px, JPEG q85
       |                               - Claude Haiku: INDOOR / OUTDOOR / INVALID
       |                               - INVALID -> 422, nothing stored
       |                               - store original in S3, row in uploaded_images
       v
5. Create the project                  POST /api/projects
       |                               status = CREATED
       v
6. Segment                             POST /api/projects/{id}/segment
       |                               - reserveAiUsage()  <- credit HELD, not spent
       |                               - enqueue on Redis reliable queue
       |                               - status = SEGMENTING, returns 202 immediately
       |
       |   [async worker]
       |     a. (opt-in) Nano Banana Pro cleans + repaints surfaces fresh white
       |     b. FLUX.2 [max] paints the photo into RED/GREEN/BLUE/BLACK blocks
       |     c. MaskProcessor.splitColorCodedMask -> 3 binary masks
       |     d. each mask smooth-upscaled to canvas resolution, uploaded to S3
       |     e. one Region row per non-empty category
       |     f. incrementAiUsage()  <- credit COMMITTED
       |        (any failure  ->  credit REFUNDED, status = FAILED)
       v
7. Frontend polls GET /api/projects/{id}/status  until SEGMENTED
       |
       v
8. THE FUN PART — all client-side, zero cost
       - pick any shade from ~8,000       WebGL2 recolour, 60 fps
       - per-region colours                every painted wall stays painted
       - click-to-segment a missed wall    SAM 2, free
       - compare slider, before/after
       - AI palette suggestions            Claude Sonnet + CIELAB ΔE snap
       - export PNG / colour-board PDF     quota-limited per tier
       - share link                        public, tokenised, expiring
```

5.2 · Project status machine



Region categories assigned by the auto path: `MAIN_WALL`, `ACCENT_WALL`, `OTHER_WALL`, `TRIM`. A region the user creates by clicking is `MANUAL` — category unknown until they label it.

5.3 · The three ways in

Route	Principal	What happens
Full account	`RETAILER` / `DISTRIBUTOR` / `PAINTER`	Email+password or Google OAuth2. Subscription and quota attach to this user.
Code → account	`CUSTOMER`	`POST /api/access-codes/redeem-account` auto-creates a **passwordless** customer account and returns a real session. The primary walk-in flow — the customer keeps their projects and can come back.
Code → guest	`GUEST` (no user row)	`POST /api/access-codes/redeem-guest` mints a JWT whose **subject is the access-code id** and whose `scope` claim is `guest`, expiring exactly when the code does. Guest work can later be *claimed* into a real account at sign-up.

> **NICE DETAIL TO BRING UP UNPROMPTED**
>
> When a guest later signs up, `maybeClaimGuestProjects()` re-points their guest projects at the new user account and drops the guest cookie — best-effort, wrapped in try/catch, because a failure there must never block the sign-in redirect. That's the kind of "the happy path must not depend on the nice-to-have" reasoning interviewers like to hear.

6 · Authentication & authorisation

> The most-asked-about module in any interview. Know the filter chain order, the token lifecycle, and *why* each decision was made.

6.1 · The Spring Security filter chain

```
[Tomcat servlet container]
      |
DisableEncodeUrlFilter
WebAsyncManagerIntegrationFilter
SecurityContextHolderFilter
HeaderWriterFilter              <- HSTS, nosniff, X-Frame-Options: DENY
CorsFilter                      <- explicit origin allow-list, never "*"
CsrfFilter                      <- DISABLED (stateless API, no cookie auth to backend)
LogoutFilter
-------------------------------------------------------
GuestAuthFilter                 <- our filter: scope=guest tokens -> ROLE_GUEST
JwtAuthFilter                   <- our filter: Bearer token -> Authentication
SensitiveEndpointRateLimitFilter <- our filter: per-IP Redis fixed window
-------------------------------------------------------
UsernamePasswordAuthenticationFilter
OAuth2AuthorizationRequestRedirectFilter
OAuth2LoginAuthenticationFilter
ExceptionTranslationFilter
AuthorizationFilter             <- evaluates authorizeHttpRequests rules
```

> **WHY THE CUSTOM FILTERS ARE INSERTED *BEFORE* `USERNAMEPASSWORDAUTHENTICATIONFILTER`**
>
> Spring Security's `addFilterBefore(filter, X.class)` needs a *core* filter class as the position anchor — you can't anchor to another custom filter. Both `GuestAuthFilter` and `JwtAuthFilter` anchor to `UsernamePasswordAuthenticationFilter`. Their relative order doesn't matter for correctness: each only acts when the `SecurityContext` is still empty, and `JwtAuthFilter` naturally ignores a guest token because no user row has the access-code's id.

6.2 · Token design

Token	Algorithm / storage	Design notes
Access token	JWT, HS256, in memory / HttpOnly cookie	Subject = user UUID, extra claim = email. Short TTL. Never persisted server-side — that's the point of a JWT.
Refresh token	Opaque random, **SHA-256 hashed** in `refresh_tokens`	7-day TTL. Only the hash is stored, so a database dump does not yield usable sessions. Migration `V2__refresh_token_hashing.sql` truncated the table when this shipped — every user was logged out once, deliberately.
Guest token	JWT, HS256, `scope=guest`	Subject = *access-code id*, not a user id. TTL is the code's own remaining validity, so guest access dies exactly with the code. The `scope` claim means it can never be mistaken for a user token.
OAuth exchange code	One-time DB row, short TTL	Google redirects with a code; `POST /api/auth/oauth2/exchange` swaps it for real tokens. Keeps JWTs out of URLs, browser history and referrer headers.
Admin 2FA code	6-digit, emailed, DB row	Admin sign-in requires a second factor *when mail is configured*. If SMTP is off, the step-up is skipped — so an operator can never lock themselves out of their own console.
Password reset / OTP	6-digit, email or SMS	Per-IP rate-limited against brute force of the 1-in-a-million space.

JWT generation (the actual code shape)

```
public String generateToken(String userId, String email) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("email", email);
    return Jwts.builder()
            .claims(claims)
            .subject(userId)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + jwtExpirationMs))
            .signWith(getSigningKey())      // HMAC-SHA256
            .compact();
}

private SecretKey getSigningKey() {
    byte[] keyBytes = Decoders.BASE64.decode(jwtSecret);
    return Keys.hmacShaKeyFor(keyBytes);    // throws if key < 256 bits
}
```

> **FAIL-FAST DESIGN**
>
> `app.jwt.secret=${JWT_SECRET}` has **no default**. The application refuses to start without it, and `docker-compose.yml` enforces the same at the compose layer with `JWT_SECRET: ${JWT_SECRET:?...}`. The service must never run on a publicly-known signing key. Rotating the secret invalidates every access *and* refresh token — a deliberate, documented consequence.

6.3 · Route access rules

Endpoint pattern	Access
`POST /api/auth/{register,login,refresh,forgot-password,reset-password,oauth2/exchange}`	Public (rate-limited)
`GET /api/shades/**`	Public — catalogue is read-only and marketing-useful
`GET /api/share/**`	Public — tokenised share links
`POST /api/access-codes/redeem-guest`, `/redeem-account`	Public (rate-limited) — the walk-in flow
`GET /api/store/*`, `POST /api/store/*/order`, `/verify`	Public — in-store kiosk; Razorpay signature is the proof
`POST /api/leads/shop`, `/verify`, `/resend`	Public — shop account request form
`POST /api/newsletter/{subscribe,unsubscribe}`	Public — an unsubscribe behind a login isn't an unsubscribe
`POST /api/billing/webhooks/**`	Public — HMAC-verified inside the service
`/api/guest/**`	`hasRole("GUEST")`
`/api/admin/**`	`hasRole("ADMIN")`
`GET /actuator/health/**`	Public, status only (`show-details=never`)
`GET /actuator/prometheus`	Authenticated unless `METRICS_PUBLIC=true` (private networks only)
everything else	`.anyRequest().authenticated()`

> **A DETAIL THAT SHOWS CARE**
>
> The default Spring Security behaviour for an unauthenticated request when `oauth2Login` is configured is a **302 redirect to Google**. For a JSON API consumed by `fetch()`, that's useless — the browser follows the redirect and the client sees an opaque CORS failure. A custom `authenticationEntryPoint` writes a clean `401 {"status":401,"error":"Unauthorized"}` instead.

6.4 · Beyond roles — two more authorisation layers

Feature gating (`@RequiresFeature`)

A distributor decides which *pages* each of its shops may use. Rather than adding a boolean column per page (a schema change every time a page ships), the grantable set is an enum `AppFeature` — `STUDIO`, `COLOR_FINDER`, `CATALOGUE`, `PRODUCTS`, `CUSTOMER_PORTAL`, `NETWORK` — stored as rows in `retailer_feature_assignments` and enforced by a `FeatureGuardInterceptor`.

Deliberately **not** grantable: dashboard, account settings and the subscription page. A shop that can't reach its own billing page could never fix a lapsed subscription; a shop with no dashboard has nowhere to land after sign-in. Revoking those would produce an unrecoverable account.

Brand gating

A distributor also decides which paint companies a shop may work with (`retailer_brand_assignments`). The catalogue a shop sees is filtered to its assigned brands — a Berger distributor's shops don't get an Asian Paints catalogue for free.

6.5 · Password & credential handling

- **BCrypt** (cost 10) via a `PasswordEncoder` bean declared in a *separate* `PasswordConfig` class — `SecurityConfig` injecting its own bean would create a circular dependency.
- **Enumeration resistance**: forgot-password responds identically whether or not the address exists.
- **Refresh-token rotation and cleanup**: `RefreshTokenCleanupService` is a scheduled job that prunes expired rows.
- **Timing**: verification, reset and 2FA codes all have short TTLs and single-use semantics.

7 · Image upload & AI classification

7.1 · The upload pipeline

```
MultipartFile
   |
   1. Size guard          Tomcat max-http-form-post-size = 12 MB
   |
   2. MIME + magic bytes  Content-Type says JPEG/PNG/WebP AND the bytes agree
   |
   3. Thumbnailator       resize to max 1024x1024, keep aspect, JPEG q85
   |                      -> ~10x fewer Claude Vision input tokens
   |
   4. Claude Haiku 4.5    "INDOOR | OUTDOOR | INVALID — one word, no explanation"
   |                      max_tokens = 10   (a hard cost ceiling)
   |
   5. INVALID -> 422, nothing is stored, no S3 write, no DB row
   |
   6. StorageService.store()   S3 (ap-south-1) or local filesystem
   |
   7. uploaded_images row      type = INDOOR | OUTDOOR
   v
ImageResponse { id, url, type }
```

TWO THINGS TO CALL OUT

Resize before classify. A modern phone photo is 12 MP. Claude Vision bills by input tokens, which scale with pixels. Downsampling to 1024 px costs one CPU-cheap Thumbnailator call and cuts the token bill roughly tenfold with *no* measurable accuracy loss for a three-way classification. This one line is the difference between ₹0.30 and ₹3 per upload.

HEIC handling. iPhones send HEIC with a JPEG Content-Type header. ImageIO can't decode it, so `UnsupportedFormatException` is caught and turned into a human error message that actually names the cause ("e.g. HEIC from iPhone") rather than a generic 500.

7.2 · Storage abstraction

`StorageService` is a 35-line interface with two implementations:

Implementation	When active	Notes
`S3StorageService`	`S3_BUCKET_NAME` set (`S3EnabledCondition`)	AWS SDK v2, ap-south-1, presigned URLs, CloudFront in front.
`LocalStorageService`	Default fallback	Filesystem. Files served through `/api/images/files/**` with an access-code-prefix authorisation check.

The condition is a Spring `Condition`, not just `@ConditionalOnProperty`, because the decision depends on more than one property being non-blank. Result: the app runs locally with zero AWS credentials, and switching to S3 is one environment variable.

8 · The paint catalogue

8.1 · Domain model

```
Brand (asian-paints, berger, dulux, nerolac, nippon)
  |
  +-- PaintLine        (product lines within a brand)
  |
  +-- Shade            name, code, hex, rgbR/G/B, LRV, shadeFamily,
  |                    temperature, tonality, styleTags[], moodTags[],
  |                    finishRecommendations[], aiDescription
  |
  +-- ShadeCodeScheme  a shop's own code convention (+ RetiredShadeCodeScheme)
  +-- ShopProduct      the shop's own product list and pricing
  +-- RetailerCombo    curated colour combinations a shop offers
```

8.2 · Two seeding strategies

	Asian Paints path	Berger / Dulux / Nerolac / Nippon path
Trigger	Admin-called: `POST /api/admin/paint/seed/asian-paints`	Automatic on startup (`BrandCatalogSeeder`)
Volume	~2,400 shades (50 in the sample file)	~8,000 shades, bundled in the JAR
Enrichment	**Yes** — each shade sent to Claude for style tags, mood, finish advice, description (~1–2 min per 50)	**No** — raw import; hex/RGB/LRV computed locally
Idempotency	De-duplicated by `(brand, code)`	Same; duplicate codes within *and across* files skipped (the two Nerolac files overlap heavily and merge into one brand)
Bad data	Validated	Rows with malformed hex dropped with a warning, not a failed startup

The bulk load is disabled with `app.catalog.auto-seed=false` — the test suite sets this so 8,000 real shades don't drown out fixtures.

8.3 · Query performance

The catalogue endpoint supports filtering by brand, family, temperature, tonality and free-text search, in any combination. That's a combinatorial query problem, solved with the **JPA Criteria / Specifications API** (`ShadeSpecifications`) rather than a dozen hand-written repository methods or string-concatenated JPQL:

```
Specification<Shade> spec = Specification
        .where(ShadeSpecifications.hasBrand(brand))
        .and(ShadeSpecifications.hasFamily(family))
        .and(ShadeSpecifications.hasTemperature(temperature))
        .and(ShadeSpecifications.matchesSearch(q));
Page<Shade> page = shadeRepository.findAll(spec, pageable);
```

On top of that, a Redis cache with per-cache TTLs:

Cache	TTL	Rationale
`shades`	60 min (configurable)	Paginated result sets — most-hit endpoint in the product
`shade-families`	6 hours	Distinct family names change only when the catalogue is reseeded
`shade-detail`	6 hours	Single shade lookups
`shade-brands`	6 hours	Brand summaries

A REAL BUG THIS DESIGN FORCED ME TO FIX

Redis cache values are serialised with the default JDK serializer, so every cached payload must be `Serializable`. `ResponseEntity` is **not**. That means a `@Cacheable` controller method must return the payload *directly*, never wrapped in a `ResponseEntity` — otherwise it fails at runtime the first time the cache tries to write. It's documented in a comment right above the cache manager so the next person doesn't rediscover it.

9 · The segmentation pipeline

> This is the technically hardest part of the project and the best material for a deep-dive interview. Everything here is a trade-off between cost, latency and quality.

9.1 · The core insight

The obvious way to build "show my room in a different colour" is to hand the photo and the colour to a generative model and let it redraw the room. I rejected that as the primary engine for three reasons:

Problem	Detail
Cost	₹3–8 per render. A customer trying twelve shades costs ₹36–96 — more than the shop's entire monthly subscription. Margin is destroyed at every tier below Enterprise.
Latency	10–30 seconds per colour change. That is unusable standing at a shop counter; the customer disengages.
Hallucination	The model subtly moves furniture, changes lighting, alters the window. The customer says *"that's not my house"* and trust is gone — which is precisely the thing the product is selling.

> THE ALTERNATIVE
>
> Use AI **once**, to answer the only question that actually needs intelligence: *"which pixels are paintable wall?"* Then do the recolouring with deterministic 2D compositing in a WebGL fragment shader.
>
> The result is photorealistic **by definition** — it IS the original photograph, with only the hue of the masked pixels replaced and the original luminance preserved. It cannot hallucinate, because nothing is being generated. And once the mask exists, every subsequent colour change is free and instant.

9.2 · The auto path, step by step

```
POST /api/projects/{id}/segment
   |
   |  [request thread]
   |  1. resolve WHO PAYS (owner vs. issuing shop for a redeemed customer)
   |  2. billingService.reserveAiUsage()      credit HELD
   |  3. segmentationJobQueue.enqueue()       Redis LPUSH
   |  4. return 202 + status=SEGMENTING       thread released immediately
   |
   v
[SegmentationQueueWorker — @Async("aiTaskExecutor")]
   |
   |  Step A  (opt-in, REPLICATE_IMAGE_CLEANER_ENABLED)
   |     Nano Banana Pro edits the photo:
   |       - removes wires, clutter, bushes, site debris
   |       - REPAINTS every painted surface to a fresh near-white (LRV ~85)
   |     -> the cleaned photo becomes the canvas. This matters enormously:
   |        masks are aligned to the SAME image the user will see, and the
   |        known-white albedo makes the photo an illumination map (see 16.3).
   |
   |  Step B  ReplicateMaskSegmenter — FLUX.2 [max] image-edit call
   |     Prompt: repaint the photo into FLAT COLOUR BLOCKS
   |       RED   = main paintable wall
   |       GREEN = accent / highlighter wall
   |       BLUE  = trim, frames, mouldings
   |       BLACK = everything else — sky, ground, stone, windows, fixtures,
   |               door panels (dark brown), metal railings (charcoal grey)
   |     Key trick: it PAINTS ONTO the real surfaces rather than drawing an
   |     abstract map, so the colour blocks stay pixel-aligned to the canvas.
   |
   |  Step C  MaskProcessor.splitColorCodedMask()  — pure Java, no AI
   |     per-pixel classification into binary masks (rules in 9.3)
   |
   |  Step D  per non-empty category:
   |     - countForeground() sanity check against a size threshold
   |     - resizeBinarySmooth() upscale to canvas resolution
   |     - upload PNG to S3
   |     - persist a Region row  (masks stored EXACTLY as the model produced
   |       them — no further post-processing)
   |
   |  Step E  billingService.incrementAiUsage()   credit COMMITTED
   |          status = SEGMENTED
   |
   +-- any throw -> refund the held credit, status = FAILED, job acknowledged
```

9.3 · Pixel classification rules

```
RED-dominant    R >= G+40  AND  R >= B+40  AND  R >= 100   -> MAIN_WALL
GREEN-dominant  G >= R+40  AND  G >= B+40  AND  G >= 100   -> ACCENT_WALL
BLUE-dominant   B >= R+40  AND  B >= G+40  AND  B >= 100   -> TRIM
near-WHITE      all channels >= 170  AND  spread <= 50      -> salvage bucket
everything else (black, ambiguous, anti-aliased edges)      -> unassigned
```

The distinct-hue scheme is chosen because **high chroma survives JPEG compression**. A scheme based on greyscale levels or subtle hues would be destroyed by the lossy round-trip through the model's output encoder.

THE WHITE-SALVAGE RULE — A REAL PRODUCTION BUG, FIXED

The model sometimes disobeys the four-colour instruction and leaves the accent surface **white** — typically because that wall is *already* white in the cleaned photo, so it "repaints" it white again. Those pixels were originally dropped, collapsing the output from three masks to two: the user got main + trim where the UI expects main, highlight and border.

Fix: when no usable GREEN accent exists, a large near-white area is *adopted* as the accent mask instead of being thrown away. A genuine green accent always wins; white is only the fallback. This is a good example to give when asked "tell me about a bug you debugged."

Other pixel-level details

- **8-connectivity** (including diagonals) wherever blobs are traced, so faint JPEG-compression gaps along a wall corner don't split one wall into two regions.
- **Foreground threshold** at combined grayscale > 127.
- `ensureWhiteForeground()` repairs the occasional inverted SAM point mask — SAM 2 sometimes returns the complement.
- **Smooth upscaling** with bilinear interpolation to the canvas resolution, so mask edges are anti-aliased and the WebGL blend has a soft boundary to work with.

9.4 · The manual path — click-to-segment

The auto path is not perfect, so there is always a manual safety net. The user clicks a point on the photo; the backend calls **Meta's SAM 2** on Replicate with that single point as a positive prompt; SAM returns the mask of the surface under the click, saved as a `MANUAL` region.

A CONCURRENCY PROBLEM I HAD TO SOLVE — GREAT INTERVIEW MATERIAL

Click-to-segment is **synchronous**: the request thread blocks while SAM 2 runs, up to ~60 s (30 poll attempts × 2 s). Tomcat has 200 worker threads. Without a cap, a burst of impatient clicking could hold every worker hostage and wedge the entire API — including login and health checks.

```
private static final int MAX_CONCURRENT_POINT_SEGMENTATIONS = 8;
private final Semaphore pointSegmentationSlots =
        new Semaphore(MAX_CONCURRENT_POINT_SEGMENTATIONS);
```

Beyond 8 concurrent segmentations we **fail fast with 503** rather than queueing more blocked threads. A bounded semaphore turns an unbounded resource leak into a bounded, observable back-pressure signal. This is a textbook bulkhead pattern and interviewers love it.

9.5 · The reliable job queue

Segmentation is long-running and expensive, so it must survive a worker crash. The Redis-backed queue implements the **reliable-queue pattern**:

```
enqueue    LPUSH   huevista:segmentation:jobs                <- pending list
dequeue    RPOPLPUSH jobs -> processing                      <- ATOMIC move
           (job is now in BOTH the worker's hand and the processing list)
acknowledge LREM   huevista:segmentation:processing          <- only after success

requeueStale   a job in `processing` longer than STALE_AFTER_MS (10 min)
               is presumed crashed -> moved back to `jobs`
               attempts counter incremented; dropped after MAX_ATTEMPTS = 3
               so a poison payload cannot loop forever
```

Property	Value / behaviour
Queue key	`huevista:segmentation:jobs`
Processing key	`huevista:segmentation:processing`
Stale threshold	10 minutes
Max attempts	3, then the job is dropped (poison-message guard)
Max queue depth	500 (`app.segmentation.queue.max-depth`) — beyond that, **503**

The depth cap is a cost control as much as a stability one: an unbounded backlog under a burst means unbounded Replicate spend. Rejecting with a "try again in a few minutes" is cheaper and more honest than silently accepting work you can't afford.

9.6 · Why "reserve then commit" instead of "charge on success"

```
reserveAiUsage()     -> quota decremented NOW, before any money is spent upstream
      |                  (so two simultaneous requests can't both pass a
      |                   "do you have quota?" check and double-spend)
      |
   [expensive AI work]
      |
      +-- success -> incrementAiUsage()   commit the hold
      +-- failure -> refund the hold      user is not charged for our outage
```

The naive alternative — check quota, do the work, then decrement — has a classic TOCTOU race: two concurrent requests both read "1 remaining", both proceed, and the shop gets two projects for one credit. Reserving first closes the window.

10 · The AI layer & colour science

10.1 · One client, many callers

`common/ai/ClaudeService` is the **single** Anthropic client. Every feature that talks to Claude — support agent, image classification, cleaning hints, shade enrichment, colour recommendations — goes through it, so the endpoint, auth headers, API version and response parsing exist in exactly one place.

It exposes two levels with *deliberately different failure semantics*:

Method	Failure mode	Used by
`complete()`	**Fail soft** — returns `Optional.empty()` on a missing key or any error	Support agent. If Claude is down, the widget degrades to a contact form; nobody's blocked.
`askUser()`	**Throws** — caller chooses semantics	Classification (fail hard: reject the upload), cleaning hints (fail soft: skip the hint).

Helper builders keep call sites readable: `textBlock()`, `imageUrlBlock()`, `imageBase64Block()`, plus `stripCodeFences()` — because Claude sometimes wraps a JSON answer in ```` ```json ```` fences despite being told not to. Defending against that in one place beats debugging it in five.

10.2 · Colour recommendations

```
POST /api/projects/{projectId}/recommendations
   |
   1. Send the room photo + the shop's assigned brands to Claude Sonnet Vision
   2. Claude returns coordinated palettes as free-form hex colours
   3. DeltaEMatcher.findNearest() snaps EVERY hex to a real catalogue shade
   4. Response carries MatchedShade { shade, deltaE } so the UI can show
      how close the real paint is to the AI's idea
```

> **WHY THE ΔE SNAP STEP EXISTS**
>
> An LLM will happily suggest `#8FA9B8`. But you cannot buy `#8FA9B8` — you can only buy shades that exist in a manufacturer's catalogue. Letting the model name shades directly invites hallucinated product codes, which is worse than useless in a shop. So the model proposes *colours*, and deterministic colour science maps them to *products*. The LLM does taste; the maths does correctness.

10.3 · The ΔE matcher — real colour science

Nearest-shade matching cannot be done in RGB, because RGB distance does not correspond to perceived difference: a numeric distance of 30 in the greens is invisible, while 30 in the blues is obvious. The correct space is **CIELAB**, which is designed to be perceptually uniform.

```
sRGB hex
   |  1. linearise each channel (undo the sRGB transfer function)
   |        c <= 0.04045 ?  c/12.92  :  ((c+0.055)/1.055)^2.4
   |
   |  2. linear RGB -> XYZ    (sRGB D65 matrix)
   |        X = 0.4124564R + 0.3575761G + 0.1804375B
   |        Y = 0.2126729R + 0.7151522G + 0.0721750B
   |        Z = 0.0193339R + 0.1191920G + 0.9503041B
   |
   |  3. XYZ -> L*a*b*  (D65 white point 0.95047, 1.00000, 1.08883)
   |        f(t) = t > 0.008856 ? cbrt(t) : 7.787t + 16/116
   |        L = 116·f(Y/Yn) − 16
   |        a = 500·(f(X/Xn) − f(Y/Yn))
   |        b = 200·(f(Y/Yn) − f(Z/Zn))
   v
CIE76 Delta E  =  sqrt(dL^2 + da^2 + db^2)
```

ΔE value	Perceptual meaning
< 1	Not distinguishable by the human eye
1 – 2	Perceptible only on close inspection
2 – 10	Perceptible at a glance
> 10	Clearly different colours

The same maths powers the **Colour Finder** feature (pull the nearest catalogue codes out of any photo) and the **competitor translator** (find the Berger equivalent of an Asian Paints shade).

> **HONEST LIMITATION TO STATE IF ASKED**
>
> This is **CIE76**, the original 1976 formula. CIEDE2000 is more perceptually accurate, especially for saturated colours and near-neutrals, at the cost of a much more complex formula with hue-rotation and chroma-compensation terms. For "find the closest paint chip out of 8,000", CIE76 was accurate enough and the difference in ranking is rarely visible. Saying *"I know CIEDE2000 exists and here's why I didn't use it"* is a much stronger answer than pretending CIE76 is the state of the art.

11 · Billing & Razorpay — the full chapter

> Payments is where interviewers probe hardest, because it is where correctness actually costs money. This chapter covers every flow, every signature check, and every failure mode.

11.1 · What Razorpay is and which products are used

Razorpay is India's dominant payment gateway — it handles cards, UPI, netbanking and wallets behind one API, and it is the practical default for an INR-denominated product. Three of its products are used:

Razorpay product	Used for	Why that product
Subscriptions	Monthly plans (Starter / Professional / Business)	Razorpay owns the recurring mandate, the retry schedule and the renewal charge. Building recurring billing on one-time Orders would mean re-implementing mandates, retries and dunning.
Orders	Buying points; the ₹99 in-store kiosk sale	One-time, no mandate needed.
Webhooks	Subscription lifecycle + refunds	Checkout verification only covers the *first* payment. Everything after — renewals, failures, cancellations, refunds — arrives by webhook.

Everything else a shop can buy — extra projects, reopens — is paid in **points** and touches Razorpay not at all. *Spending* points involves no Razorpay object; *buying* them is an ordinary Order.

11.2 · Configuration surface

```
RAZORPAY_KEY_ID=rzp_test_xxxxxxxxxxx    # sent to the browser ON PURPOSE — Checkout needs it
RAZORPAY_KEY_SECRET=...                 # NEVER leaves the server; signs and verifies
RAZORPAY_WEBHOOK_SECRET=...             # separate secret, HMAC for webhook bodies
RAZORPAY_PLAN_STARTER=plan_xxxxxxxxxxx
RAZORPAY_PLAN_PROFESSIONAL=plan_xxxxxxxxxxx
RAZORPAY_PLAN_BUSINESS=plan_xxxxxxxxxxx
```

> **THE KEY ID / KEY SECRET DISTINCTION — A GUARANTEED INTERVIEW QUESTION**
>
> The **Key ID** is public by design: the browser needs it to open Checkout, and it is embedded in the client bundle. The **Key Secret** is what proves a payment is real. It must never appear in the Next.js repository or in any `NEXT_PUBLIC_*` variable — anything with that prefix is inlined into the JavaScript bundle at build time and is therefore world-readable. In this project the secret exists only in the Spring Boot process's environment.

The bean, and why it doesn't crash on boot

```
@Bean
public RazorpayClient razorpayClient() throws RazorpayException {
    if (keyId.isBlank() || keySecret.isBlank()) {
        log.warn("Razorpay credentials not configured — billing features will be unavailable");
        return new RazorpayClient("", "");   // calls fail at runtime with clear messages
    }
    return new RazorpayClient(keyId, keySecret);
}
```

Deliberate: a developer running the app locally to work on the catalogue shouldn't need payment credentials. The failure is deferred to the moment billing is actually used, with a readable message — rather than a startup crash that blocks unrelated work.

11.3 · Flow A — Subscription purchase (the full sequence)

WHY VERIFY AT CHECKOUT *AND* HANDLE THE WEBHOOK — BOTH

Checkout verification makes the plan active *the instant the buyer finishes paying*, without leaving the app. If you relied on the webhook alone, the buyer would pay and then stare at a "pending" screen for anywhere between two seconds and two minutes — and would email support.

The webhook is the durable, authoritative channel. It covers every event the browser cannot report: renewals in month two, a failed card, a cancellation from the Razorpay dashboard, a refund, or the buyer closing the tab at exactly the wrong moment. Both paths converge on the same idempotent `activateSubscription()`.

11.4 · Business guard rails on subscription creation

Before a single call goes to Razorpay, `createSubscription()` refuses several requests. Each one exists because it would otherwise take money for nothing:

Guard	Reason
Plan is `ENTERPRISE`	Custom-priced (`-1`), quoted not sold. There is no plan ID to charge against.
Plan is `FREE`	Granted, never sold. Letting it through would create a Razorpay subscription for ₹0.
User role is `CUSTOMER`	Plans are shop products. A customer's access is governed by the shop-issued access code, which a subscription doesn't touch — they'd pay and unlock nothing. The error message redirects them: *"redeem the code your paint shop gave you instead."*
Already on that exact plan	Prevents accidental double-subscription.
Requested plan is **not an upgrade** from an active paid plan	Only an upgrade may happen in place. A downgrade or sideways move must cancel first, so nobody double-pays for a lateral change. `isUpgradeFrom()` compares enum `ordinal()` — the declaration order *is* the tier ladder.
A pending attempt already exists for this plan	`reusePendingAttempt()` hands back the existing attempt instead of opening a second subscription object at the gateway, and retires attempts for plans the shop has moved on from.

THE CANCELLATION TRAP I HAD TO FIX

Originally, a subscription already scheduled to end still blocked new purchases — so after cancelling, the still-`ACTIVE` row rejected both re-subscribing to the same plan *and* moving to a smaller one. The only way back was to wait for the period to expire. The fix is one filter: `.filter(s -> !s.isCancelAtPeriodEnd())`. A plan winding down is no longer treated as a plan in force.

Scheduled starts — not double-charging

If a shop upgrades mid-cycle, the old plan has already been paid for through the end of its period. Billing the new plan immediately would take a second month's money *and* throw away the remaining paid days when the old row was superseded. So `scheduledStartFor(userId)` computes the day the current period ends and passes it to Razorpay as `start_at`. The new subscription row carries that start date from the moment it exists, so `findEntitling()` can distinguish "bought, starts later" from "in force now."

Quantity scaling

`quantity` multiplies what Razorpay bills, so the quotas scale with it — otherwise a shop paying 3× would still get a single plan's limit. Note the asymmetry: `projectsLimit` and `pdfDownloadsLimit` scale; `pdfImageLimit` does **not**, because images-per-document is a per-document property of the tier (and a browser-memory guard), not an allowance.

11.5 · Signature verification — the two kinds

(a) Checkout signature — proves a payment happened

```
Subscription:  HMAC_SHA256( razorpay_payment_id + "|" + razorpay_subscription_id, KEY_SECRET )
Order:         HMAC_SHA256( razorpay_order_id   + "|" + razorpay_payment_id,      KEY_SECRET )
compare against razorpay_signature   (constant-time)
```

Without this, a hostile client could simply POST a made-up payment id to `/verify` and be granted a plan. The signature is unforgeable without the secret, which is why the secret must be server-only.

(b) Webhook signature — proves the event came from Razorpay

```
private void verifySignature(String payload, String signature) {
    if (webhookSecret == null || webhookSecret.isBlank()) {
        // FAIL CLOSED. This endpoint is permitAll. With no secret we cannot
        // trust the payload, and accepting unsigned events would let ANYONE
        // forge subscription.activated / payment.captured to grant free plans.
        log.error("Razorpay webhook secret not configured — rejecting webhook.");
        throw new SecurityException("Webhook signature verification is not configured.");
    }
    if (!Utils.verifyWebhookSignature(payload, signature, webhookSecret)) {
        throw new SecurityException("Invalid Razorpay webhook signature");
    }
}
```

> **"FAIL CLOSED" IS THE WHOLE ANSWER TO A COMMON INTERVIEW QUESTION**
>
> Asked *"what happens if the webhook secret isn't configured?"*, the tempting answer is "we log a warning and process it anyway, so we don't lose events." That is a critical vulnerability: the webhook endpoint is `permitAll`, so anyone on the internet could POST a forged `subscription.activated` and grant themselves a Business plan. The correct behaviour is to **reject every webhook** until the secret exists. Losing events is recoverable; granting free plans to attackers is not.

Note the raw body is taken as `@RequestBody String payload`, not a parsed object. HMAC is computed over the *exact bytes* Razorpay signed — deserialising and re-serialising would reorder keys or change whitespace and break verification.

11.6 · Idempotency — the hardest correctness problem in payments

Razorpay retries webhook deliveries with exponential backoff whenever it doesn't get a 2xx. That means **the same event will arrive more than once**, and a captured request could be replayed within its signature validity window. Processing `payment.captured` twice would credit two months of quota for one payment.

```
if (StringUtils.hasText(eventId)) {
    if (processedEventRepository.existsById(eventId)) {
        log.info("Razorpay webhook already processed, skipping: {}", eventId);
        return;
    }
    processedEventRepository.save(ProcessedWebhookEvent.builder().eventId(eventId).build());
} else {
    log.warn("Razorpay webhook without X-Razorpay-Event-Id — processing without replay protection");
}
```

Three properties make this correct, and each is worth stating explicitly:

1. **The marker insert shares the handler's `@Transactional` boundary.** If processing fails halfway, the marker rolls back with it, so the retry is processed cleanly rather than being skipped as "already done."
2. **A duplicate racing insert dies on the primary key.** Two concurrent deliveries of the same event: one wins, the other gets a constraint violation, returns 5xx, and Razorpay's retry then finds the marker and skips. The *database* is the concurrency control, not application-level locking.
3. **Missing event id is logged loudly** rather than silently trusted.

11.7 · Every webhook event handled

Event	Handling
`subscription.activated`	`activateSubscription(id, charge_at, current_end)` — plan goes live, quotas provisioned, older plans superseded.
`subscription.charged`	**The authoritative renewal event.** Carries the real `current_end`, so period dates stay exact instead of drifting off a 30-day approximation.
`payment.captured`	**Fallback** renewal path if `subscription.charged` isn't delivered. Passes `0` so the service falls back to +30 days — and it will *never shrink* a period end that `subscription.charged` already set accurately.
`subscription.cancelled`	`markCancelled()` — terminal.
`subscription.completed`	`markCompleted()` — the mandate ran its `total_count`. Terminal.
`subscription.halted`	`markHalted()` — Razorpay gave up after repeated payment failures. Terminal.
`refund.created` / `refund.processed`	`walletService.reverseKioskPayment(paymentId, amount)` — a pay-then-chargeback must not leave the shop's reward points standing.
`payment.failed`	Informational, logged with reason and description. The lifecycle is driven by `subscription.halted`, which carries the subscription id; this is logged so a spike is visible without digging through the dashboard.
anything else	`log.debug` — unknown events must not throw, or Razorpay retries forever.

Terminal statuses

```
private static final Set<SubscriptionStatus> TERMINAL_STATUSES =
        EnumSet.of(EXPIRED, CANCELLED, COMPLETED);
```

Reaching one of these means the gateway will *never charge this subscription again*, so nothing may re-activate it in place — a new subscription has to be bought. Encoding that as a set beats scattering three-way status comparisons through the service.

11.8 · HTTP status codes are part of the protocol

```
catch (SecurityException e) {
    return ResponseEntity.status(401).body(Map.of("error", "Invalid signature"));
} catch (Exception e) {
    // Return 5xx so Razorpay RETRIES (its webhooks use exponential backoff).
    // Returning 200 here would SILENTLY DROP a genuine event — a transient DB
    // failure during activation would permanently lose that transition.
    log.error("Webhook processing error: {}", e.getMessage(), e);
    return ResponseEntity.status(500).body(Map.of("status", "error", "message", "Processing failed"));
}
```

> **THIS IS THE SINGLE BEST PAYMENTS ANSWER IN THE WHOLE PROJECT**
>
> The instinct when a handler throws is to catch it and return 200 so the caller stops complaining. With webhooks, **200 means "I have durably handled this"** — it tells Razorpay to stop retrying and forget the event. If your database was briefly unavailable, you have now permanently lost a real customer's activation, and no amount of log-reading will replay it.
>
> Returning 500 enlists Razorpay's own exponential-backoff retry as your recovery mechanism. Combined with the idempotency marker, retries are free: at-least-once delivery plus exactly-once processing.
>
> Note also that the raw exception message is never echoed to the caller — it could leak schema or infrastructure detail to an unauthenticated endpoint.

11.9 · The safety-net job

Webhooks can be delayed or lost entirely. A daily `@Scheduled` sweep hard-expires any PAID subscription whose period has lapsed — but only after a **3-day grace window** (`RENEWAL_GRACE_DAYS`). Without the grace period, a webhook delayed by an hour would cut off a customer who has genuinely paid. This is defence in depth: the webhook is the fast path, the job is the backstop, and the grace window is the apology for the gap between them.

`razorpay.subscription.total-count` defaults to **120** — ten years of monthly cycles — so an ongoing plan doesn't silently auto-complete after twelve months, which is the default trap.

11.10 · The points ledger

Points are a **closed-loop currency** — the only balance in the product. There is no prepaid rupee wallet beside it and no per-item cash checkout; both existed once and were removed so that every purchase has exactly one price in exactly one unit.

Property	Value
Purchase rate	₹1 = 1 point (`app.points.rupees-per-point`)
Purchase bounds	min 100, max 100,000 per order
Earned at kiosk	30 points per ₹99 walk-in sale
Validity	365 days, with a 10-day expiry warning email
Accounting	`RewardPointsLot` (FIFO lots with expiry) + `RewardPointsTransaction` (immutable ledger)
Eligibility	**Retailers only** — enforced in the service, not a controller, so no future caller can route around it
Cash-out	**Impossible by design** — there is no payout method on the class and there must not be one

> **A COMPLIANCE DECISION, EXPLAINED AS AN ENGINEERING CONSTRAINT**
>
> Converting points to a bank transfer would make every kiosk sale *"a payment collected on the shop's behalf"* — which in Indian payments regulation makes you a payment aggregator, a licensed activity. So the kiosk price is flat and platform-set (₹99), the shop takes **no cash share**, and it is rewarded in closed-loop points instead. The shop cannot price the link and keep the excess. The absence of a payout method is a deliberate, documented architectural constraint, not an unimplemented feature.
>
> Bought and earned points are deliberately **indistinguishable** once credited — same expiry clock, same prices — so the ledger never has to answer "which kind of point was that", which is what would make a refund, an expiry or a spend ambiguous.

11.11 · Money representation

Every cash amount in the system is an `int` of **paise** (₹1 = 100 paise), matching Razorpay's own API. No `float`, no `double` — floating-point cannot represent 0.1 exactly, and accumulated error in a billing system is a support ticket you cannot answer.

```
public int priceWithTaxInPaise() {
    if (priceInPaise < 0) return -1;                    // -1 = custom pricing
    return priceInPaise * (100 + GST_PERCENT) / 100;
}
```

`GST_PERCENT` is currently `0` — the project runs as an individual, non-GST-registered entity, so prices are billed and shown flat. It's a single constant, so restoring 18% is a one-line change… *plus* recreating the Razorpay plans, because **plan amounts cannot be edited after creation**. That coupling is documented right where it bites.

A KNOWN GAP I'D FIX NEXT — SAY THIS BEFORE THEY FIND IT

The application **does not verify** that the amount configured on the Razorpay dashboard plan equals the price it shows the customer. If someone creates the Professional plan at ₹2,000 in the dashboard while `Plan.java` says ₹2,499, the pricing page advertises one number and Razorpay charges another — a chargeback and a gateway-review problem. The fix is a startup check that fetches each plan via the API and asserts the amount matches. Naming an unimplemented safeguard *before the interviewer finds it* reads as ownership, not as a gap.

12 · Accounts, hierarchy & access codes

12.1 · The organisation model

```
Organization (type = DISTRIBUTOR | RETAILER)
   |
   +-- OrgMembership (user, role = OWNER | MEMBER)
   |
   +-- DistributorRetailerLink        distributor -> its shops
   +-- RetailerBrandAssignment        which paint companies this shop may sell
   +-- RetailerFeatureAssignment      which product pages this shop may use
   +-- CustomerAccessCode             codes the shop hands to walk-ins
   +-- CustomerEntitlement            what a redeemed customer is entitled to
   +-- ProjectGrant                   specific projects granted to a code/customer
```

12.2 · The access-code system

A walk-in customer must be able to use the product without an account, without a card, and without the shop losing control of its spend. That is what access codes do.

Property	Behaviour
Format	8-character generated code
Validity	3 / 7 / 14 days — the shop chooses at issue time
Funding	A project **credit is held against the code** at issue. The shop's quota is committed the moment it hands out the code, so it can never over-issue.
Redemption — account	`POST /api/access-codes/redeem-account` auto-creates a passwordless `CUSTOMER` and returns a full session. The primary flow.
Redemption — guest	`POST /api/access-codes/redeem-guest` mints a guest JWT scoped to the code, expiring with it.
Top-up	A shop can add credits to an already-issued code (`AccessCodeTopUpIntegrationTest`).
Role guard	Only retailer-side roles may generate codes (`AccessCodeRoleGuardTest`).
Brute-force defence	Redemption endpoints are per-IP rate-limited — an 8-character space is guessable at scale, and a guessed code *burns a real shop's credit*.

> **THE "WHO OWNS" VS "WHO PAYS" SPLIT — A GENUINELY SUBTLE PIECE OF DESIGN**
>
> When a customer redeems a shop's code and creates a project, the project is **owned** by the customer (it's their room, their data, they keep it) but **billed** to the shop that issued the code, spending the credit that code is holding.
>
> Those are two different questions and they are resolved by two different components: `ProjectAccessService` answers *"may this principal see this project?"* and `ProjectBillingResolver` answers *"whose credit does this run spend?"*. Conflating them is how you end up billing the wrong party — or leaking one customer's room photo to another.

12.3 · Time-boxed access

A redeemed code opens a **window**, not a permanent grant. When the window closes, the customer keeps their account but loses access to the project until someone reopens it — a shop can buy a reopen for 9 points, or a purchased project credit opens 30 days (`app.project-credit.valid-days`).

> **TIMEZONE GOTCHA — WORTH MENTIONING AS AN OPERATIONAL LESSON**
>
> The schema stores **zone-naive** timestamps (`LocalDateTime`). Every share link, subscription period and access-code expiry is therefore interpreted in the JVM's default zone. The Docker image pins `TZ=Asia/Kolkata` and `docker-compose.yml` repeats it, because moving the JVM to a UTC host would silently shift every expiry by 5½ hours. Given the chance again I would store `Instant` / `timestamptz` and format per-user — but pinning the zone was the correct *cheap* fix for a single-country product, and knowing the difference is the point.

12.4 · The rest of the network

Module	What it does
`hierarchy/`	Distributor → retailer → painter network, brand and feature assignment, and a downline report (`NetworkReportResponse`). Access control is integration-tested end to end (`DistributorAccessControlIntegrationTest`).
`painter/`	Painter profiles, invitations, painter↔retailer links, and paint jobs. A shop creates painter accounts under itself.
`store/`	Public kiosk links (`/store/{slug}`), Razorpay order creation and verification, and the wallet that reverses on refund.
`lead/`	Public "request a shop account" form with email verification, plus `ShopRequestAutoApprovalJob` — if an admin doesn't act within 24 hours, the account is provisioned automatically. A queue that only drains when a human is looking is not a queue.
`support/`	Claude-backed support agent with conversation history, plus inbound channel webhooks (WhatsApp, ElevenLabs).
`newsletter/`	Subscribe / unsubscribe. Both public: most readers have no account, and an unsubscribe behind a login is not an unsubscribe. The unsubscribe token is the authorisation and can only remove its own address.
`admin/`	Admin console, audit log, user search, shade upload, and a guarded platform data reset (`DataResetTransaction`).

13 · Cross-cutting concerns

13.1 · Rate limiting

`SensitiveEndpointRateLimitFilter` is a per-IP, Redis-backed **fixed-window** limiter (INCR + EXPIRE) covering every sensitive endpoint:

Endpoint	Threat
`POST /api/auth/register`	Bulk account creation
`POST /api/auth/login`	Credential stuffing / password spraying
`POST /api/auth/refresh`	Token grinding
forgot / reset password	Reset-email bombing + 6-digit OTP brute force
OTP send (email / SMS)	Message bombing — costs real money via Twilio
OTP confirm	6-digit code brute force
access-code redeem	8-char code brute force — griefing a shop by burning its credits
subscribe / verify	Gateway subscription spam + Checkout-payload replay
image upload	Bounds the Claude Vision bill (this endpoint is not quota-charged)
kiosk order / verify	Public endpoints; one shop's customers share an IP

Three design decisions inside the limiter

> **1 · FAIL *OPEN*, NOT CLOSED**
>
> If Redis is unreachable, requests are allowed through. This is the **opposite** of the webhook decision (11.5) and the contrast is the interesting part: a rate limiter is a *availability* control, so a limiter outage must never lock legitimate users out of logging in. A signature check is an *integrity* control, so its outage must never let forged events through. **Same question, opposite answers, because the thing being protected is different.** If an interviewer asks about only one, volunteer the other.

> **2 · TRUSTED CLIENT IP DERIVATION**
>
> Naively reading `X-Forwarded-For` means anyone can rotate a fake IP per request and bypass every limit. `ClientIps` counts trusted proxy hops **from the right** — `app.rate-limit.trusted-proxy-hops` (default 1) — so only the hop your own proxy appended is trusted. And `RATE_LIMIT_TRUST_FORWARDED=false` exists for deployments where the backend port is reachable directly.
>
> The frontend cooperates: `middleware.ts` **overwrites** `X-Forwarded-For` with its own trusted derivation before rewriting `/api/auth/*` to the backend, and the BFF route sets it explicitly — otherwise every request would bucket under the Next.js server's single IP and one user could exhaust the limit for everybody.

> **3 · HONEST 429S**
>
> Over-limit responses carry `429` *and* a `Retry-After` header, so a well-behaved client knows exactly when to try again rather than hammering.

13.2 · Exception handling

A `@RestControllerAdvice` `GlobalExceptionHandler` maps twelve domain exception types to consistent JSON. Every one of them exists so the frontend can branch on *meaning* rather than parsing English out of a message string:

Exception	Maps to	Frontend reaction
`ResourceNotFoundException`	404	Not-found state
`QuotaExceededException`	402 / 429	"Buy more" upsell
`ProjectLimitReachedException`	402	Show extra-project price
`SubscriptionRequiredException`	402	Route to /pricing
`AccessExpiredException`	403	Offer a reopen
`VerificationRequiredException`	403	Route to the verify step
`RetailerActionRequiredException`	403	"Ask your shop to…"
`ImageValidationException`	422	Explain what to upload instead
`ExternalServiceException`	503	Retry affordance
`StorageException`	500	Generic failure
`ProcessingInterruptedException`	500	Retry

13.3 · Audit logging

`AuditService` writes an immutable `audit_log` row for every consequential action — plan changes, admin operations, data resets, access-code issuance, credit adjustments. Surfaced in the admin console. In a system where an admin can grant a subscription or reset platform data, "who did that and when" must be answerable without reading application logs.

13.4 · Load management

Every layer has a bound, so a spike degrades gracefully rather than falling over:

Setting	Default	Protects against
`server.tomcat.threads.max`	200	Thread explosion under load
`server.tomcat.accept-count`	100	Unbounded accept backlog
`server.tomcat.max-connections`	8192	Memory exhaustion
`server.tomcat.connection-timeout`	20s	Slowloris / stalled clients holding workers
`spring.datasource.hikari.maximum-pool-size`	20	Overwhelming PostgreSQL
`hikari.connection-timeout`	10s	Threads blocking forever on a pool
`MAX_CONCURRENT_POINT_SEGMENTATIONS`	8	SAM 2 calls wedging all Tomcat workers
`app.segmentation.queue.max-depth`	500	Unbounded backlog → unbounded AI spend
`server.shutdown=graceful`	25s	Killing in-flight requests on deploy

> **THE PRINCIPLE BEHIND ALL OF THESE**
>
> Every queue must have a maximum depth, every pool a maximum size, every blocking call a timeout. An unbounded resource is a resource that will be exhausted — and when it is exhausted, the failure is total rather than partial. Bounding turns "the site is down" into "some requests got a 503 and retried."

13.5 · Configuration diagnostics

`ConfigDiagnosticRunner` runs at startup and reports which optional integrations are actually live — mail, SMS, S3, Razorpay, Replicate, Claude. It's the difference between "the emails aren't sending and nobody knows why" and a line in the boot log saying mail is disabled. It has its own unit test.

14 · Next.js App Router architecture

14.1 · Route structure

```
src/app/
├── page.tsx                  marketing home — STATIC RSC, edge-cacheable
├── layout.tsx  template.tsx  root shell, fonts, theme
├── middleware.ts  (in src/)  route gate + token refresh
├── catalogue/  pricing/  method/  trial/  join/  legal/*
│                             public marketing — static
├── sign-in/                  form, Google route handler, callback, forgot
├── redeem/                   access-code redemption
├── share/[token]/            PUBLIC share view — no auth
├── store/[slug]/             PUBLIC in-store kiosk
├── m/[id]/                   phone hand-off upload (desktop -> phone camera)
├── (app)/                    ROUTE GROUP — authenticated shell
│   ├── layout.tsx            app nav, plan banner, support widget
│   ├── dashboard/  atelier/  catalogue tools/  color-finder/
│   ├── portal/  products/  assigned-products/  network/  inbox/
│   ├── account/  subscription/
│   └── admin/                admin console, shades, mask-viewer, migration
├── bff/[...path]/route.ts    server-side auth proxy
└── api/handoff/*             Next-native routes for the phone hand-off
```

The `(app)` parenthesised segment is a **route group**: it applies a shared layout (nav, plan banner, support widget) to every authenticated page without adding `/app` to any URL.

14.2 · Rendering strategy

Page class	Strategy	Why
Marketing	Static RSC, edge-cacheable	Sub-second first paint; these pages are the SEO and conversion surface.
App pages	Streaming SSR + Suspense	Shell renders instantly, data streams in. No spinner-on-blank-page.
Atelier (the studio)	Client component	Needs WebGL, canvas and continuous interaction.
Share / kiosk	Public SSR	Must work with no session at all.

14.3 · The middleware — two jobs in one place

`middleware.ts` lives in `src/` (not the project root) so Next actually runs it for a `src/`-based app — a real gotcha that silently disables the whole file if you get it wrong.

Job 1 — route gating

```
const PROTECTED_PREFIXES = ["/atelier", "/dashboard", "/portal", "/inbox",
  "/products", "/assigned-products", "/color-finder", "/account",
  "/admin", "/subscription", "/network"];
const GUEST_ONLY_PATHS  = ["/sign-in", "/sign-in/forgot", "/join"];
```

> **A DUPLICATION NEXT.JS FORCES ON YOU — AND HOW IT'S DEFENDED**
>
> Next requires `config.matcher` to be a **static literal**, so the same route list necessarily exists twice in the file. A route missing from either copy stops refreshing its access cookie and starts bouncing people to `/sign-in` — a bug that looks like an auth failure but is a config typo.
>
> Rather than rely on a comment, there is a test (`protected-routes.test.ts`) that holds **both** lists against the pages that actually exist under `(app)`. When the language forces duplication, make a test the source of truth.

Job 2 — token refresh

This is the more interesting one, and it's the fix for a real crash.

```
Server Components MUST NOT mutate cookies during render.
Doing so throws: "Cookies can only be modified in a Server Action or Route Handler."

BEFORE: refresh happened lazily during render  ->  crash on every expired token
AFTER:  refresh happens in middleware, which runs BEFORE render, where
        cookies ARE writable.

  middleware:  no hv_access but hv_refresh present
                 -> POST /api/auth/refresh
                 -> res.cookies.set("hv_access", newToken, {httpOnly, secure, sameSite:lax})
                 -> continue to render, which now only READS
  getAccessToken(): READ-ONLY, by contract, documented in a comment
```

15 · The BFF pattern & token handling

15.1 · The threat this closes

The conventional SPA approach stores a JWT in `localStorage` and attaches it in JavaScript. That means **any** XSS — one bad dependency, one unescaped string — reads the token and exfiltrates a full session. Token theft becomes a one-line payload.

HueVista's browser JavaScript **never sees a token at all.**

```
Browser                    Next.js server                Spring Boot
   |                            |                            |
   | fetch("/bff/api/projects") |                            |
   | (no Authorization header;  |                            |
   |  the cookie is HttpOnly,   |                            |
   |  JS cannot read it)        |                            |
   |--------------------------->|                            |
   |                            | 1. isBffPathAllowed(path)?  |
   |                            | 2. read hv_access from the  |
   |                            |    server-side cookie jar   |
   |                            | 3. Authorization: Bearer ... |
   |                            | 4. X-Forwarded-For: real IP  |
   |                            |----------------------------->|
   |                            |<-----------------------------|
   |<-- body streamed back, only safe headers passed through --|
```

Cookie	TTL	Holds
`hv_access`	token expiry	Access JWT — HttpOnly, Secure, SameSite=Lax
`hv_refresh`	7 days	Refresh token — HttpOnly, Secure, SameSite=Lax
`hv_guest`	code validity	Guest JWT for walk-ins with no account

15.2 · Defence in depth inside the proxy

Control	What it prevents
Path allow-list (`isBffPathAllowed`)	The proxy is not an open relay. Only paths the API client actually calls are forwardable. Kept in `lib/bff-paths.ts` so a test can hold the list against every path `api.ts` calls.
Header filtering	Only `Content-Type` , `Accept` , `Authorization` and `X-Forwarded-For` go upstream. Response passes back only `content-type` , `content-length` , `cache-control` , `etag` — no upstream server banners or internal headers leak.
Token selection by scope	`/api/guest/*` uses the guest cookie, everything else the user cookie, so a signed-in user and a guest session stay cleanly separate. One documented fallback: a guest with no user session still needs non-guest GETs like `/api/images/files/**` to see their own photo — the backend authorises the guest principal against the file's access-code prefix, so handing it the guest token is safe.
`redirect: "manual"`	The proxy never silently follows an upstream redirect to somewhere unexpected.
Error opacity	An upstream connection failure logs internally and returns a bare `502` `{"message":"Upstream unreachable"}` — the internal origin/host is never disclosed to the browser.
Streaming body	`duplex: "half"` streams request bodies (image uploads) rather than buffering them in the Node process.

15.3 · CSRF

Backend CSRF is disabled because the API is stateless and does not authenticate by cookie. On the frontend, mutating forms go through **Next.js server actions**, which require a same-origin request — that is the CSRF defence. Combined with `SameSite=Lax` cookies and a `form-action 'self'` CSP directive, cross-site form submission is closed off at three layers.

16 · The WebGL2 recolour engine

> 536 lines of TypeScript and GLSL. This is the piece that makes the product economically viable and the piece most worth being able to explain at the shader level.

16.1 · What it does

Upload the source photograph to the GPU **once**, then composite any number of per-region masks over it in a single frame. Every painted wall stays painted while you edit another one — a naive implementation redraws from the source per region and produces the classic "switching tabs wipes the last colour" bug.

```
Pass 0 (base):   draw the photo, u_useMask = 0, forced opaque
                 (so an exported PNG never has see-through holes)
Pass 1..N:       for each region, bind its mask texture, set u_target
                 to the shade's RGB, draw a full-screen quad with blending
Result:          one frame, N painted regions, ~60 fps on mid-range mobile
```

16.2 · The naive approach and why it fails

```
Naive:  outColor = mix(photo, targetColour, mask)

The wall becomes a flat rectangle of colour. Every shadow, every soft
gradient across the wall, every bit of plaster texture and every corner
falls off a cliff. It looks like a paint-bucket fill in MS Paint —
which is exactly what it is — and a customer will not believe it.
```

16.3 · What the shader actually does

The photograph is split into two layers, and the swatch is reassembled from them:

```
FORM   = a low-pass (blurred) copy of the photo  -> large-scale light:
         eaves, reveals, the wall's own gradient, the light falling across it

DETAIL = photo − blur                            -> high-frequency truth:
         plaster stipple, dirt, seams, micro-shadows, brush texture

paint  = target_colour  ×  FORM  +  DETAIL
```

That is why the result reads as *real paint on a real wall*: the surface's actual texture is carried onto whatever colour the user picks. A flat fill can't fake it and synthetic grain looks synthetic.

Uniforms, and what each one is for

Uniform	Purpose
`u_image`	Source photograph texture
`u_blur`	Low-pass copy — the "form" light layer
`u_mask`	Current region's binary mask (soft-edged)

`u_target`	The shade's RGB — the colour in the can
`u_preserve`	**0** = flat exact fill (true swatch everywhere, matches the can); **1** = fully follow the photo's light. Dialable per region.
`u_baseL`	The region's mean luminance — its effective LRV in this photo
`u_anchor`	Scene-light anchoring mode (see below)
`u_grain`	A hair of per-pixel hash noise — a texture floor for perfectly smooth walls where the photo itself carries almost no detail
`u_bright`	Whole-image gamma midtone lift, `output = input^(1/u_bright)`. Applied to the base photo **and** the painted regions alike, so paint sits in the same brightened light instead of floating dark on a lifted photo. Gamma (not linear gain) keeps pure white where it is, so bright skies don't clip.

The two normalisation modes — the cleverest part

```
if (u_anchor > 0.5) {
  // ANCHORED: the canvas is the CLEANED image, whose paintable surfaces are
  // known fresh white (LRV ~85, REF_WHITE = 0.94). So the smooth photo IS
  // the illumination map. Divide by the known albedo to recover the scene's
  // light — per channel, so a warm or cool cast tints the paint too.
  form = Brgb / REF_WHITE;
} else {
  // LEGACY: normalise by the region's own mean luminance, so the wall still
  // averages to the true swatch colour (the colour in the can).
  form = vec3(B / u_baseL);
}
```

WHY THE ANCHORED MODE EXISTS — CONNECT THIS BACK TO THE PIPELINE

Because the clean-up step repaints every paintable surface a known fresh white, the photograph of those surfaces stops being "a picture of a wall" and becomes **a measurement of the light falling on it** — level and colour cast both. Dividing by the known white albedo recovers the illumination directly.

The practical difference: an evening photo keeps its dim, warm evening light instead of snapping to flat noon daylight. A pipeline decision two steps upstream (repaint to a reference white) unlocked a rendering capability downstream. Being able to trace that dependency is exactly the kind of systems thinking a senior interviewer is listening for.

Highlight and shadow handling

```
form = clamp(form, 0.30, 2.4);
form = mix(vec3(1.0), form, u_preserve);

// Channels below 1: multiply with a mild gamma so genuine shadows DEEPEN and
// the paint sits INTO the surface instead of floating flat on top.
vec3 dark = u_target * pow(min(form, vec3(1.0)), vec3(1.0 + 0.25 * u_preserve));

// Channels above 1: lift toward white, ROLLED OFF so a bright wall never clips
// to pure white (a hard clip read as blown-out white patches).
vec3 lift = max(form - 1.0, 0.0);
lift = (lift / (lift + 0.7)) * 0.7;
```

Each half equals `u_target` where it's inactive, so summing them and subtracting `u_target` composes the two per channel **without a branch** — branches are expensive in a fragment shader that runs millions of times per frame.

16.4 · Mask edge handling

A hard mask edge produces a visible jagged seam where paint meets window frame. Two utilities fix that:

- `featherMaskInward` — softens the mask boundary *inward* only. Feathering outward would bleed paint onto the window frame; inward keeps the paint strictly inside the wall and lets the anti-aliased edge do the blending.
- `featherRadiusInMaskPx` — computes the feather radius in mask-space pixels, so the softness is consistent whether the mask was stored at 512 px or 2048 px.
- `mask-grow.ts` — the inverse, for masks that undershoot the true surface.

The mask's soft edge is **the only place colour blends**. Everywhere else the fill is exact.

16.5 · Graceful degradation

`recolor-engine.ts` defines a `RecolorEngine` interface with two implementations: the WebGL2 engine, and a 351-line **Canvas 2D fallback** for devices without WebGL2. Lower fidelity, but it works. In a market where a meaningful share of shop counters run older Android tablets, "no WebGL2" cannot mean "no product."

17 · Frontend security posture

17.1 · Content Security Policy

```
default-src  'self'
script-src   'self' 'unsafe-inline' https://checkout.razorpay.com
style-src    'self' 'unsafe-inline' https://fonts.googleapis.com
font-src     'self' https://fonts.gstatic.com data:
img-src      'self' data: blob: {apiHost} https://*.s3.{region}.amazonaws.com
connect-src  'self' https://*.razorpay.com https://lumberjack.razorpay.com
frame-ancestors 'none'
frame-src    https://api.razorpay.com https://checkout.razorpay.com
base-uri 'self'   form-action 'self'   object-src 'none'
upgrade-insecure-requests
```

Directive	Reasoning
No `'unsafe-eval'` in production	Dev-only, for HMR. Its absence closes a large class of injection escalation.
`connect-src` excludes the API origin in production	The browser only ever talks to its own origin via `/bff/*`. The public API origin is deliberately **not** listed — an injected script cannot even name the backend.
`frame-ancestors 'none'`	Clickjacking. Paired with `X-Frame-Options: DENY` for older browsers.
S3 host keyed by region	CSP supports only one wildcard label, so `https://*.s3.ap-south-1.amazonaws.com` is the tightest expressible form.
Razorpay hosts enumerated exactly	Checkout needs a script, a frame and a connect target. Each is whitelisted individually rather than opening a category.

17.2 · Other headers

Header	Value
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload` (production only)
`X-Frame-Options`	`DENY`
`X-Content-Type-Options`	`nosniff`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(self), microphone=(), geolocation=(), interest-cohort=(), payment=(self "https://checkout.razorpay.com")`
`Cross-Origin-Opener-Policy`	`same-origin`
`Cross-Origin-Resource-Policy`	`same-origin`

AN SSRF I CLOSED IN THE IMAGE OPTIMISER — STRONG MATERIAL

Next.js's `/_next/image` optimiser fetches and caches remote images server-side. A blanket `{ hostname: "**" }` in `remotePatterns` turns it into an **open proxy**: an attacker names any HTTPS host and your server fetches it — including cloud metadata endpoints and internal services behind your perimeter. Classic SSRF.

The optimiser is therefore restricted to exactly the same hosts `img-src` allows: the API host, the configured S3 region, and any explicitly listed extras. The lesson generalises: **any server-side fetch driven by a client-supplied URL is an SSRF until you constrain it.**

18 · Database design & migrations

18.1 · The 43 entities by module

Module	Entities
`auth`	`User`, `RefreshToken`, `VerificationCode`, `PasswordResetCode`, `AdminLoginCode`, `OAuthExchangeCode`
`account`	`Organization`, `OrgMembership`, `DistributorRetailerLink`, `CustomerAccessCode`, `CustomerEntitlement`, `ProjectGrant`, `RetailerBrandAssignment`, `RetailerFeatureAssignment`
`billing`	`Subscription`, `SubscriptionPayment`, `ProcessedWebhookEvent`, `ProjectCredit`, `ProjectPurchase`, `PointsPurchase`, `RewardPointsLot`, `RewardPointsTransaction`
`paint`	`Brand`, `PaintLine`, `Shade`, `ShadeCodeScheme`, `RetiredShadeCodeScheme`, `ShopProduct`, `RetailerCombo`
`project`	`Project`, `Region`
`image`	`UploadedImage`
`painter`	`PainterProfile`, `PainterInvitation`, `PainterRetailerLink`, `PaintJob`
`store`	`StoreLink`, `StorePayment`
`support`	`Conversation`, `SupportMessage`
`lead` / `newsletter` / `common`	`ShopLead`, `NewsletterSubscriber`, `AuditLog`

18.2 · Flyway owns the schema — the discipline

Rule	Why
Flyway applies migrations at startup	The schema is a versioned artefact, reviewable in a pull request.
`ddl-auto=validate` in production	Hibernate **checks** that the schema matches the entities and never mutates it. Entity/schema drift is caught at boot, not at 3 a.m. when a query hits a missing column.
`V1__baseline.sql` is never edited	Generated from the JPA entities against PostgreSQL 16 at the moment Flyway was adopted.
`baseline-on-migrate=true`	Databases predating Flyway (created by the old `ddl-auto=update`) are adopted automatically: a non-empty schema is stamped as already at V1, and only V2+ is applied. Zero manual migration steps.
Every change = a new versioned file	Change the entity **and** write the migration. `validate` at boot catches you if you forget.
Dev (H2) and tests disable Flyway	The migrations are PostgreSQL SQL. Dev/test use Hibernate DDL generation instead.
`MigrationsCoverEntitiesTest`	A test that asserts the migrations actually cover the entities — the discipline is machine-enforced, not just documented.

The 37 migrations tell the product's story

```
V1   baseline                    V20  access code assignment
V2   refresh token HASHING       V21  user provider access code
V3   shop leads                  V22  image holds and reversals
V4   project sent to shop        V23  retailer feature assignments
V5   oauth exchange codes        V24  project access windows
V6   admin login codes (2FA)     V25  shop show shade names
V7   store kiosk                 V26  project grants
V8   retailer combos             V27  self-funded access codes
V9   shade code scheme           V28  admin grants without distributor
V10  pdf quota                   V29  kiosk reward points
V11  project raw mask            V30  reward points LEDGER
V12  project share brands        V31  points-only billing
V13  project skip image clean    V32  subscription replay + carryover
V14  project mask enhancements   V33  single project quota
V15  DROP mask enhancements  <-- V34  retired shade code schemes
V16  subscription tiers rework   V35  subscription plan free tier
V17  billing wallet              V36  newsletter subscribers
V18  user hierarchy              V37  shop account requests
V19  retailer brand assignments
```

V14 THEN V15 — SAY THIS OUT LOUD IN AN INTERVIEW

V14 added mask enhancements; V15 dropped them. That is a feature that was built, measured, and removed because it didn't earn its keep. Migrations are an honest changelog: **V17 "billing wallet" → V31 "points-only billing"** is a whole billing model that was replaced with a simpler one. Being able to point at your own reversals reads as engineering maturity, not as churn.

18.3 · Schema-level integrity

Two reconciler components — `ImageTypeConstraintReconciler` and `RegionCategoryConstraintReconciler` — keep database `CHECK` constraints in sync with the Java enums. When you add a value to `RegionCategory`, the constraint that would otherwise reject it is updated too. Enum values enforced in *both* the application and the database means invalid data cannot enter through any path, including a manual SQL fix.

19 · Infrastructure, Docker & CI/CD

19.1 · Containerisation

A multi-stage `Dockerfile`: Maven build stage, slim JRE runtime stage, non-root user, `TZ=Asia/Kolkata` pinned, and a `HEALTHCHECK` against `/actuator/health`.

`docker-compose.yml` brings up a prod-like local stack — PostgreSQL 16 + Redis 7 + backend — with:

- `depends_on: condition: service_healthy` so the app doesn't start before its dependencies are actually ready (not merely started).
- `env_file: .env` so any backend variable is settable without editing the compose file.
- `JWT_SECRET: ${JWT_SECRET:?Set JWT_SECRET in .env}` — compose **fails fast** rather than starting a container on a missing key.
- A named volume for Postgres data.

19.2 · CI

GitHub Actions runs `./mvnw verify` on every pull request, with H2 in-memory for tests so no external infrastructure is needed in CI. The frontend runs `npm run lint`, `npm run typecheck` and `npm test` (Vitest). Dependabot is active — the git history shows merged dependency PRs.

19.3 · Observability

Concern	Setup
Health	`/actuator/health` public, status only; `/health/liveness` and `/health/readiness` for orchestrators. Used by both the Docker `HEALTHCHECK` and load balancers.
Metrics	Micrometer → Prometheus. Inert until `ACTUATOR_EXPOSURE=health,prometheus` **and** `METRICS_PUBLIC=true`. JVM, HTTP latency/status, Hikari pool, cache stats.
Alerting targets	5xx rate, p95 latency, connection-pool exhaustion, heap.
Logs	`logs/huevista.log`, rotating 10 × 10 MB. Production runs at `INFO` — `DEBUG` emits PII (user emails) and infrastructure detail (S3 bucket/object keys).

19.4 · Production checklist (from the README)

1. **Email + SMS** — these flags also drive the retailer verification gate: a channel is only *required-verified* when it can actually deliver a code, so leaving them off silently weakens onboarding. Two sender addresses (`no-reply@`, `payments@`) must both be SPF/DKIM/DMARC-authorised on one domain.
2. **Admin bootstrap** — `ADMIN_EMAIL` / `ADMIN_PASSWORD` is a *login identity, not a shared inbox*; lead notifications go to `LEADS_EMAIL` separately.

3. **Razorpay** — create the three plans, set the IDs, point a webhook at `{APP_BASE_URL}/api/billing/webhooks/razorpay`. Renewals depend on it.
4. **Network posture** — `RATE_LIMIT_TRUST_FORWARDED=false` if the backend port is reachable except through your own proxy.
5. **Timezone** — keep the JVM on IST everywhere.
6. **Monitoring** — Prometheus scrape on a private network only.
7. **Backups** — `pg_dump` schedule + S3 bucket versioning. Masks and cleaned images are reproducible at cost; originals are not.
8. **Swagger OFF** — the spec and UI are public when enabled and enumerate the whole API surface.

20 · Testing strategy

20.1 · Coverage shape

439	53	40
BACKEND @TEST METHODS	BACKEND TEST CLASSES	FRONTEND VITEST SUITES

Layer	Approach
Integration (`*IntegrationTest`)	`@SpringBootTest` + MockMvc against H2. Full request→DB→response path with real Spring Security. Covers auth, billing webhooks, project flow, hierarchy, access codes, guest flow, painter flow, shop leads, admin, images, AI recommendations.
Unit	Plain JUnit + Mockito for pure logic: `MaskProcessorTest`, `ProjectAccessPolicyTest`, `ClientIpsTest`, `DeltaE` maths, quota arithmetic.
Contract	`ApiContractTest` — pins the shape of the API so a rename doesn't silently break the frontend.
Schema	`MigrationsCoverEntitiesTest` — migrations must cover the entities.
Serialization	`ShadeResponseSerializationTest`, `ShadeSummaryResponseSerializationTest` — the JSON shape is itself a contract.
Frontend	Vitest + Testing Library on jsdom: components, hooks, route guards, form validation.

20.2 · Tests that encode business rules, not just code paths

The value of the suite is that the test names read like a specification of the money model:

Test	Business rule it protects
`BillingWebhookIntegrationTest`	Signature verification, idempotent replay, every event type
`UnlimitedPlanQuotaOverflowTest`	`Integer.MAX_VALUE` as "unlimited" must not overflow when scaled by quantity — a genuine arithmetic trap
`ProjectHoldAccountingTest`	Reserve → commit / refund accounting is exact
`BillingSubscriptionVerifyTest`	Checkout signature verification
`BillingServiceLifecycleTest`	Full state machine: created → active → cancelled/halted/completed
`AccessCodeRoleGuardTest`	Only retailer-side roles may issue codes
`ImageControllerKeyGuardTest`	One customer cannot read another's uploaded photo
`SensitiveEndpointRateLimitFilterTest`	Limits apply; Redis outage fails *open*
`ConfigDiagnosticRunnerTest`	Boot-time integration reporting is accurate
`StubAiPipelineTest`	The free-testing mode still charges credits, so limits stay testable

20.3 · How externals are handled

Every external service — Razorpay, Replicate, Claude, S3, SMTP, Twilio — is mocked in tests (`application-test.properties`). Tests are hermetic: no network, no API keys, no cost, deterministic. That is what lets CI run the full suite on every pull request.

21 · Key decisions & trade-offs

Decision	Alternative rejected	Reasoning
2D WebGL recolour	Generative AI re-render	₹3–8 and 10–30 s per render, plus hallucinated furniture. 2D is free, instant, and photorealistic by construction. Generative stays a Premium-tier feature for styled final renders.
Colour-coded mask from one image-edit call	N separate segmentation calls	One call returns all three surface classes. Because the model paints *onto* real surfaces rather than drawing an abstract map, the blocks stay pixel-aligned to the canvas.
Project as the billable unit	Per-API-call metering	Two quotas could run out independently and strand a user with a cleaned photo and no way to mask it. One unit, one price, all-or-nothing.
BFF proxy + HttpOnly cookies	JWT in `localStorage`	Closes XSS token theft entirely. Costs a network hop through the Next server.
Flyway + `ddl-auto=validate`	`ddl-auto=update`	`update` is unreviewable, irreversible, and silently drifts. Flyway makes the schema a code-reviewed artefact.
Feature-sliced packages	Layer-sliced (`controllers/`, `services/`)	A billing change touches one directory. Module boundaries become the seams for future extraction.
Monolith	Microservices	Solo developer, pre-revenue. Microservices would add deployment, tracing and consistency problems in exchange for benefits I don't yet need. The boundaries are drawn so extraction is possible when it is.
Blocking servlet stack	WebFlux / reactive	The workload is DB + outbound HTTP, and the expensive calls are pushed off the request thread anyway. Reactive would add real complexity for a throughput ceiling I'm nowhere near.
Redis reliable queue	Kafka / RabbitMQ / SQS	Redis was already in the stack for cache and rate limiting. RPOPLPUSH gives at-least-once delivery, which is enough. Adding a broker would be an operational cost with no matching benefit at this scale.
Rate limiter fails *open*	Fail closed	Availability control. A limiter outage must not lock users out.
Webhook verification fails *closed*	Fail open	Integrity control. Unsigned events would let anyone grant themselves a plan.
Plain CSS + design tokens	Tailwind / CSS-in-JS	No build-time or runtime cost, full control of the theming system, and no framework churn. Costs some authoring convenience.
Points, not a rupee wallet	Prepaid cash balance + shop revenue share	A cash-out path would make every kiosk sale a collection on the shop's behalf — a regulated payment-aggregator activity. Closed-loop points sidestep that entirely.

22 · Hard problems solved

1 · THE "SWITCHING TABS WIPES THE LAST COLOUR" BUG

Symptom: paint one wall, switch to the second, and the first goes back to grey.
Cause: the renderer redrew from the source photograph for each region, so only the last region survived.
Fix: restructure the engine to upload the photo once and composite *all* region masks in a single frame with blending. One base pass plus N blended passes.
Lesson: the bug was in the render architecture, not the shader maths. Fixing it needed a model change, not a patch.

2 · THE DISAPPEARING ACCENT WALL

Symptom: some rooms produced two masks instead of three, and the UI's "highlight wall" control had nothing to bind to.
Cause: the model was told to paint accent walls GREEN, but when the wall was *already white* it "repainted" it white — and white pixels were being discarded as unassigned.
Fix: a near-white salvage bucket. If no usable GREEN region exists, adopt a large near-white area as the accent. A genuine green always wins.
Lesson: when a probabilistic component is upstream, the deterministic component below it has to be written to tolerate plausible disobedience — not just correct output and clear errors.

3 · THREAD STARVATION FROM SYNCHRONOUS SAM 2 CALLS

Symptom: under a burst of click-to-segment, the entire API became unresponsive — including login and health checks.
Cause: each call blocks a Tomcat worker for up to 60 s. Enough concurrent clicks exhaust all 200.
Fix: a `Semaphore(8)` bulkhead. Beyond 8 concurrent segmentations, fail fast with 503.
Lesson: a slow dependency is a resource leak unless you bound your concurrency against it. Rejecting work you can't do is better than accepting work you can't finish.

4 · COOKIES-DURING-RENDER CRASH

Symptom: every user with an expired access token hit *"Cookies can only be modified in a Server Action or Route Handler."*
Cause: token refresh was happening lazily inside a Server Component during render, where cookie mutation is forbidden.
Fix: move refresh into `middleware.ts`, which runs before render where cookies are writable; make `getAccessToken()` read-only by contract.
Lesson: framework constraints are architectural constraints. The App Router's render/mutation split isn't a limitation to work around — it tells you where the work belongs.

5 · THE CANCELLATION TRAP

Symptom: after cancelling, a shop could neither re-subscribe to the same plan nor move to a smaller one until the period expired.
Cause: the "do you already have an active plan?" guard treated a plan scheduled to end as a plan in force.
Fix: `.filter(s -> !s.isCancelAtPeriodEnd())`.
Lesson: a one-line fix for a bug that took real thought to characterise. Subscription state is not a single boolean — "active", "active but winding down" and "expired" are three different things and only two of them should block a purchase.

6 · DOUBLE-CHARGING ON MID-CYCLE UPGRADE

Symptom: upgrading mid-month took a second month's money *and* discarded the days already paid for.
Cause: the new subscription billed immediately while activation superseded (expired) the old row.
Fix: compute `scheduledStartFor(userId)` and pass it to Razorpay as `start_at`; the new row carries its start date from creation so entitlement queries can tell "bought, starts later" from "in force now".
Lesson: in billing, the *time dimension* of every state change has to be modelled explicitly. "Active" always needs a "from when."

7 · REDIS CACHE SERIALIZATION

Symptom: the first cache write on a `@Cacheable` endpoint blew up at runtime.
Cause: JDK serialization requires `Serializable` values; `ResponseEntity` isn't.
Fix: cached controller methods return the payload directly, never a `ResponseEntity` — documented in a comment above the cache manager.
Lesson: a constraint that lives in one component's configuration but is violated in a completely different file needs to be written down where it will be read.

8 · RATE LIMITING BEHIND A PROXY

Symptom: every request bucketed under one IP (the Next.js server's), so one user could exhaust the limit for everyone.
Cause: the backend saw the proxy's IP, not the visitor's.
Fix: the frontend forwards a *trusted* `X-Forwarded-For`; the backend counts trusted hops from the right rather than believing the leftmost value. And `middleware.ts` **overwrites** the header before rewriting `/api/auth/*`, because Next's rewrite proxy forwards it exactly as the client sent it — a forged header would otherwise sidestep every per-IP limit.
Lesson: `X-Forwarded-For` is client-controlled input. Treat it as hostile and configure how much of it you trust.

23 · Resume bullets & how to present the project

23.1 · The resume entry

COPY-PASTE READY — FULL VERSION (5 BULLETS)

HueVista — AI Paint Visualisation SaaS | Spring Boot 4, Java 17, PostgreSQL, Redis, Next.js 16, React 19, TypeScript, WebGL2, AWS S3, Razorpay

- Architected and built a full-stack B2B SaaS platform for the Indian paint retail trade — **78K LOC across two repositories**, 207 REST endpoints, 43 JPA entities and **479 automated tests**.
- Designed a hybrid AI pipeline that uses vision models **once per project** for wall segmentation (Claude Haiku classification, FLUX.2 colour-coded masking, Meta SAM 2 click refinement) and performs unlimited recolouring in a custom **WebGL2 luminance-preserving fragment shader** — cutting per-interaction cost from ~₹5 to **₹0** and latency from 10–30 s to **60 fps**.
- Implemented end-to-end **Razorpay** billing — Subscriptions and Orders, HMAC-SHA256 Checkout and webhook verification, **idempotent event processing** keyed on delivery ID, and an atomic reserve-then-commit quota ledger that automatically refunds credits on pipeline failure.
- Engineered a **zero-token-in-browser** auth architecture: JWT + rotating SHA-256-hashed refresh tokens, Google OAuth2, role-based access across 5 roles, and a Next.js BFF proxy with HttpOnly cookies that eliminates XSS token theft; hardened with CSP, HSTS and Redis-backed per-IP rate limiting.
- Built for operational resilience — Flyway-owned schema across **37 migrations** with `ddl-auto=validate`, a Redis reliable job queue (RPOPLPUSH) with stale-job recovery and poison-message limits, semaphore bulkheads against thread starvation, and Prometheus/Actuator observability.

COMPACT VERSION (2 BULLETS, FOR A ONE-PAGE RESUME)

- **HueVista** — Full-stack AI paint-visualisation SaaS (Spring Boot 4 · Next.js 16 · PostgreSQL · Redis · AWS S3 · WebGL2). Built a hybrid pipeline using vision models once per project for wall segmentation, then a custom WebGL2 luminance-preserving shader for unlimited free recolouring — **~₹5 → ₹0 per interaction, 30 s → 60 fps**. 78K LOC, 207 endpoints, 479 tests.
- Implemented Razorpay Subscriptions + Orders with HMAC webhook verification and idempotent event processing; a BFF architecture keeping JWTs entirely out of the browser; Flyway-versioned schema (37 migrations) and Redis-backed rate limiting, caching and reliable job queueing.

23.2 · Which skill each part of the project demonstrates

Claim on your resume	Point at this
Backend / Java / Spring	17 feature modules, 34 controllers, custom security filters, `@Async` + `@Scheduled`, JPA Specifications, interceptors, `@RestControllerAdvice`
System design	Reserve/commit ledger, reliable queue, bulkheads, bounded pools, cache TTL tiers, fail-open vs fail-closed reasoning

Security	BFF pattern, HttpOnly cookies, hashed refresh tokens, HMAC verification, CSP, SSRF fix in the image optimiser, trusted-proxy IP derivation
Payments	Two Razorpay products, both signature types, idempotency, 5xx-for-retry, grace windows, paise arithmetic
AI/ML integration	Multi-model orchestration, prompt design under cost constraints, fail-soft vs fail-hard, ΔE grounding of LLM output
Frontend	App Router, RSC vs client boundaries, streaming SSR, server actions, middleware, 40 Vitest suites
Graphics / maths	GLSL fragment shader, form/detail decomposition, CIELAB colour science, mask feathering
DevOps	Multi-stage Docker, compose with health gating, GitHub Actions, Flyway, Prometheus, graceful shutdown
Product sense	Business-model design, unit economics, regulatory-aware architecture (no cash-out), features removed after measurement (V14 → V15)

23.3 · How to run the conversation

Time you get	What to say
30 seconds	What it is, who pays, and the one architectural insight: AI once for the mask, WebGL forever for the colour.
2 minutes	Add the three pillars: segmentation pipeline, Razorpay billing, BFF security. Give one number per pillar.
10 minutes	Draw the architecture diagram. Walk the request path. Let them pick a box to go deeper on.
Deep dive	Have three stories ready: the accent-wall salvage bug, the semaphore bulkhead, and the webhook 5xx decision. Each has a symptom, a cause, a fix and a lesson.

> THE SINGLE MOST EFFECTIVE MOVE IN THIS INTERVIEW
>
> **Always give the trade-off, not just the choice.** "I used Redis for the queue" is a fact. "I used Redis because it was already in the stack for caching and rate limiting, RPOPLPUSH gives me at-least-once delivery which is enough here, and adding Kafka would have been operational cost with no benefit at this scale — if throughput grew past a few hundred jobs a minute I'd revisit it" is an engineer.

24 · Question bank — Project & architecture

Q **Tell me about this project.**

A Use the 60-second pitch from section 1. Structure: what it is → who pays → the one insight → the three engineering pillars → a number. Then stop and let them choose where to go. Do not narrate the whole feature list.

Q **Why did you build it? What problem does it solve?**

A "Choosing a wall colour is the slowest step in a paint purchase. The customer is holding a 2 cm swatch and being asked to imagine 400 square feet. They can't, so they hesitate and often leave without buying. For the shop that's a conversion problem, and shops will pay monthly for a tool that closes it. So I sell to shops, and shops hand it to walk-ins — which also solves customer acquisition, because a shop has this conversation twenty times a day while a homeowner paints once every seven years."

Q **Walk me through the architecture.**

A Draw the diagram from section 4.1. Four tiers: Next.js frontend (which also acts as a BFF), Spring Boot API, data tier (PostgreSQL + Redis + S3), and external services (Anthropic, Replicate, Razorpay). Then trace one request end to end — the request path in section 4.3 is the script.

Q **Why two separate repositories instead of a monorepo?**

A "Different languages, different build tools, different deploy targets, different release cadences. The backend deploys as a Docker image behind a load balancer; the frontend deploys to a Node/edge runtime. A monorepo would have given me shared tooling and atomic cross-stack commits — which I did miss when changing an API shape. The contract between them is defended by two things: `ApiContractTest` on the backend pins the response shapes, and an 879-line `types.ts` on the frontend mirrors the DTOs. If I were adding a mobile client I'd extract those into a shared schema package, probably OpenAPI-generated."

Q **Why a monolith? Would you use microservices?**

A "Not at this stage. I'm one developer pre-revenue. Microservices would buy me independent scaling and deployment, and cost me distributed transactions, network failure modes, distributed tracing and N deployment pipelines. The costs are immediate and the benefits are hypothetical."

"What I did instead is draw the boundaries as if I might split it. The packages are feature-sliced, not layer-sliced, so each module owns its full vertical stack. If I did extract something, the first would be `project/` — the segmentation pipeline — because it's the only module with a fundamentally different resource profile: long-running, latency-tolerant, and bursty against a third-party API. It's already behind a Redis queue, so the interface is a message boundary rather than a method call. That's a day of work, not a rewrite."

Q What was the hardest technical decision?

A "Rejecting generative AI as the recolouring engine. It was the obvious approach — everybody's demo does it, and it's more impressive in a screenshot. But at ₹3–8 and 10–30 seconds per render, and with the model subtly moving the customer's furniture, it fails on cost, latency and trust simultaneously."

"The alternative was harder to build: use AI once to answer the only question that needs intelligence — which pixels are paintable wall — then do the recolouring myself with deterministic compositing. That meant writing a GLSL shader, understanding luminance decomposition, and building a whole mask pipeline. But it's ₹0 and 60 fps, and it's photorealistic by construction because it IS the original photo with the hue replaced. Generative AI is still in the roadmap — as a Premium feature for styled final renders, where the cost is justified and the customer expects an interpretation rather than a photo."

Q What would you do differently if you started over?

A Have three real answers, in increasing order of seriousness:

- "Store `Instant` / `timestamptz` instead of `LocalDateTime`. I pinned the JVM timezone to IST as a cheap fix, which works for a single-country product, but it's a landmine the day I deploy outside India."
- "Adopt Flyway from commit one. I ran `ddl-auto=update` early, and adopting Flyway later meant a baseline migration and a `baseline-on-migrate` dance for existing databases. It worked, but starting with versioned schema costs nothing."
- "Define the API contract with OpenAPI first and generate the TypeScript types, instead of hand-maintaining `types.ts` in parallel with the DTOs. It's the one place where the two-repo split genuinely costs me."

Q How long did it take and how did you scope it?

A Answer honestly with your real timeline, then show the phasing: "Phase 1 was the MVP — upload, classify, segment, recolour, save, share, and Razorpay subscriptions with quota enforcement. Phase 2 added the distributor/retailer hierarchy, access codes and AI recommendations. Phases 3–5 are roadmap: painter accounts and ordering, depth-aware rendering, and true 3D via Gaussian Splatting if demand ever justifies it. The git history and the migration list show that sequence — `V18__user_hierarchy` is exactly where Phase 2 starts."

Q What's the most complex part of the codebase?

A "The segmentation service, at 872 lines — and the complexity is *essential*, not incidental. It has to coordinate two paid third-party models, split billing between the project owner and the shop that's actually paying, hold and commit or refund a credit atomically, run asynchronously off a durable queue with crash recovery, and degrade to a stub mode for free testing. Every one of those is a genuine requirement. The pixel logic is factored out into `MaskProcessor`, the Replicate protocol into `ReplicateMaskSegmenter`, and billing resolution into `ProjectBillingResolver`, so what's left is the orchestration itself."

Q How do you handle failures in the AI pipeline?

A "Layer by layer. The credit is *held*, not spent, before any upstream call — so a failure refunds automatically and the customer isn't charged for my outage. The project moves to `FAILED` rather than hanging in `SEGMENTING`, so the UI can offer a retry. The queue job is only acknowledged after the work completes, so a JVM crash mid-job leaves the payload in the processing list and `requeueStale` puts it back after ten minutes. And attempts are capped at three so a poison payload can't loop forever."

"Different callers also get different semantics deliberately: classification fails *hard*, because storing an unclassified image would corrupt downstream assumptions. The support agent fails *soft* — it returns `Optional.empty()` and the widget degrades to a contact form. Neither is universally right; it depends on whether the feature is load-bearing."

Q What are the current limitations?

A Answer this straight — evasion reads worse than any limitation:

- "Segmentation quality depends on a probabilistic model. Cluttered rooms and unusual architecture produce imperfect masks. That's why click-to-segment exists and why it's free and unlimited — the manual path isn't a fallback I'm embarrassed by, it's a designed part of the flow."
- "It's 2D. There's no depth, so I can't do parallax or render a wall from a different angle. Depth Anything v2 is on the roadmap for that."
- "No load testing under real concurrency. The bounds are reasoned rather than measured — I've capped pools and queues everywhere, but I haven't run k6 against it to find the actual knee."
- "Single-region deployment, no read replicas. Fine for one country and this traffic; not a global architecture."
- "And one I'd fix first: the app doesn't verify that the Razorpay dashboard plan amount matches the price in `Plan.java`. If they drift, I advertise one number and charge another."

Q If traffic went 100×, what breaks first?

A "The segmentation pipeline, and it breaks on *cost* before it breaks on capacity — 100× the runs is 100× the Replicate bill, and my queue depth cap would start shedding load at 500. That cap is deliberately a cost control as much as a stability one."

"Technically, the order would be: (1) the Replicate rate limit and spend, (2) the Hikari pool at 20 connections, (3) Tomcat's 200 workers if synchronous SAM 2 calls scaled with traffic — though the semaphore bounds that at 8, so it'd shed rather than starve, (4) PostgreSQL write throughput on the audit and ledger tables."

"The fixes in order: horizontally scale the API (it's stateless — sessions are JWTs and the queue is external), move segmentation workers to their own autoscaling pool, add read replicas for the catalogue, and put PgBouncer in front of Postgres. None of that needs a rewrite, which is the point of having drawn the boundaries where I did."

25 · Question bank — Java & Spring Boot

Q Why Spring Boot? Why version 4?

A "Spring Boot gives me security, data access, validation, scheduling, caching, metrics and health checks as configuration rather than code, with one dependency-management story. Boot 4 (Spring Framework 7, Security 7) was current when I started."

"One Boot 4 specific I hit: auto-configuration is split per technology module now. Adding `flyway-core` alone does nothing — you need the `spring-boot-flyway` module for the auto-configuration to wire up. That cost me an afternoon and it's documented in a comment in my `pom.xml` so the next person doesn't repeat it."

Q Explain dependency injection as you've used it.

A "Constructor injection everywhere, via Lombok's `@RequiredArgsConstructor` on `final` fields. Three reasons over field injection: the object is fully initialised and immutable after construction; the compiler tells me when a class has too many dependencies, which is a design smell I'd otherwise not notice; and the class is testable with plain `new` in a unit test — no reflection, no Spring context."

"One place I use `@Autowired(required = false)` deliberately: `SegmentationJobQueue` in `SegmentationService`. The queue only exists when Redis is configured, and the service has a direct-async fallback when it isn't. That's an optional collaborator, and marking it as such is honest about the wiring."

Q What Spring annotations have you used and what do they actually do?

Annotation	What it does here
`@RestController`	`@Controller` + `@ResponseBody` — return values are serialised to JSON, no view resolution
`@Service` / `@Repository` / `@Component`	Stereotypes for component scanning. `@Repository` additionally translates persistence exceptions into Spring's `DataAccessException` hierarchy
`@Transactional`	Proxy-based transaction boundary. Used with `readOnly=true` on queries (lets Hibernate skip dirty-checking) and `Propagation.REQUIRES_NEW` where a credit refund must survive the rollback of the run that failed
`@Async("aiTaskExecutor")`	Runs segmentation on a named executor off the request thread
`@Scheduled`	Subscription safety-net sweep, refresh-token cleanup, points expiry, shop-lead auto-approval
`@Cacheable`	Redis-backed catalogue caching with per-cache TTLs
`@RestControllerAdvice`	Global exception → JSON mapping
`@ConditionalOnProperty` / custom `Condition`	Redis cache manager only when `spring.cache.type=redis`; S3 only when a bucket is configured
`@Value`	Property binding with defaults — `${razorpay.key-id:}` allows blank, `${JWT_SECRET}` deliberately doesn't
`@EnableMethodSecurity`	Enables `@PreAuthorize` alongside the URL rules

Q How does `@Transactional` actually work, and where does it fail?

A "Spring creates a proxy around the bean. Calls that come in *through the proxy* get a transaction opened before and committed or rolled back after. The classic failure is a **self-invocation**: if method A calls method B on `this`, the call never goes through the proxy, so `@Transactional` on B does nothing. The fix is to move B to another bean or self-inject."

"The second gotcha is rollback rules: by default Spring rolls back on `RuntimeException` and `Error`, but **not** on checked exceptions. That's caught out plenty of people who catch, wrap and rethrow."

"In my webhook handler the boundary matters directly: the idempotency marker insert shares the transaction with the event processing, so if processing fails, the marker rolls back too and Razorpay's retry gets processed cleanly rather than being skipped as already-done."

Q How do you handle asynchronous work?

A "Two mechanisms for two different needs."

"`@Async` with a named executor (`AsyncConfig`) for fire-and-forget work off the request thread. Important: I configure the executor explicitly rather than using the default `SimpleAsyncTaskExecutor`, which spawns an unbounded new thread per task — that's a production incident waiting to happen."

"A **Redis-backed durable queue** for anything that must survive a restart. `@Async` holds work in an in-memory queue: if the JVM dies, that work is gone and the project hangs in `SEGMENTING` forever. The Redis reliable-queue pattern — RPOPLPUSH to a processing list, acknowledge only on completion, requeue stale jobs after ten minutes — means a crash costs a retry, not a lost job. Expensive third-party work with money attached to it deserves durability."

Q What Java 17 features did you use?

- **Records** — DTOs, and internal value types like `Policy` and `Rule` inside the rate-limit filter, and `ClaudeService.Turn`. Immutable, with `equals` / `hashCode` / `toString` free.
- **Pattern-matching `switch` with arrow syntax** — the whole webhook event dispatcher is a `switch` expression, no fall-through, exhaustive.
- **`instanceof` pattern matching** — `if (!(text instanceof String s) || s.isBlank())` in the Claude response parser.
- **Text blocks** — multi-line AI prompts stay readable instead of being string-concatenated.
- **`var`** — locally, where the type is obvious from the right-hand side.
- **Enhanced enums** — `Plan` carries seven fields, computed pricing methods, and uses `ordinal()` as the upgrade ladder.

Q Why did you make `Plan` an enum instead of a database table?

A "Because plans are **code**, not data. Each tier carries seven attributes — price, project limit, PDF image cap, PDF download limit, extra-project point price, extra-project cash price, display name — plus behaviour: `priceWithTaxInPaise()`, `isUpgradeFrom()`, `isFree()`. Adding a tier means changing pricing logic, adding a Razorpay plan in the dashboard and updating the pricing page. That's a deploy, not a database row."

"There's a nice property in it: `isUpgradeFrom()` compares `ordinal()`, so **declaration order is the tier ladder**. FREE sits at the bottom deliberately, so every paid tier reads as an upgrade from it. If plans were rows I'd need an explicit rank column that someone could set inconsistently."

"The honest downside: I can't change pricing without a deploy, and I can't run per-customer custom pricing. If I needed either — say, enterprise contracts negotiated individually — I'd move to a table with the enum kept as the default catalogue."

Q How do you handle configuration and secrets?

A "Everything is an environment variable read through `${VAR:default}` placeholders in `application.properties`. The default is the design decision: optional integrations default to off (`razorpay.key-id:` allows blank and degrades gracefully), while anything security-critical has **no default at all** — `app.jwt.secret=${JWT_SECRET}` means the app refuses to boot without it, and `docker-compose.yml` repeats the check with `${JWT_SECRET:?...}` so `docker compose up` fails fast too."

"Secrets never enter the repository — `.env` is gitignored, and `.env.example` documents every variable with prose about what it does. There's also a `ConfigDiagnosticRunner` that logs at startup which integrations are actually live, so 'the emails aren't sending' is a line in the boot log rather than a mystery."

"For a real production deployment I'd move to AWS Secrets Manager or Parameter Store with rotation. Environment variables are visible to anything that can read `/proc` and end up in crash dumps."

Q What is Lombok doing and what are its risks?

A "It's an annotation processor that generates boilerplate at compile time: `@RequiredArgsConstructor`, `@Getter`, `@Builder`, `@Slf4j`. It's declared `optional` in the POM and excluded from the Spring Boot fat jar, because it's only needed at compile time."

"Risks I'm aware of: it's a compiler hack that depends on internal javac APIs, so a JDK upgrade can break it; `@Data` on a JPA entity generates `equals`/`hashCode` over all fields, which breaks with lazy proxies and can trigger loading an entire object graph — I use `@Getter` and explicit builders on entities rather than `@Data`; and it hurts newcomers reading the code because the constructor isn't visible."

Q How would you profile and optimise a slow endpoint?

A "Measure first. Micrometer already exposes per-endpoint HTTP latency percentiles, so I'd start at the p95 in Prometheus rather than guessing."

"Then the usual suspects in order: (1) N+1 queries — turn on `spring.jpa.show-sql` or use the Hibernate statistics and count the queries per request; fix with `JOIN FETCH` or an entity graph. (2) Missing indexes — `EXPLAIN ANALYZE` the actual SQL. (3) Serialisation of over-large payloads — the catalogue endpoint returns paged summaries, not full entities, for exactly this reason. (4) Blocking calls on the request thread — which is why segmentation is async and click-to-segment is semaphore-bounded."

"For the catalogue specifically, the answer was caching: it's read-heavy, the data changes only on reseed, so Redis with a 60-minute TTL on result sets and 6 hours on the near-static lookups removes most of the load without any query tuning at all."

Q Explain the difference between `@Component`, `@Service`, `@Repository` and `@Configuration`.

A "All four are component-scanned into beans. `@Service` and `@Component` are functionally identical — the distinction is documentation. `@Repository` adds real behaviour: exception translation from JDBC/JPA vendor exceptions into Spring's `DataAccessException` hierarchy. `@Configuration` is different in kind — it's CGLIB-proxied so that calling one `@Bean` method from another returns the singleton rather than constructing a second instance."

Q How do you validate incoming requests?

A "Bean Validation via `spring-boot-starter-validation`: `@Valid` on the controller parameter and constraints on the DTO. Violations surface as `MethodArgumentNotValidException`, which `GlobalExceptionHandler` maps to a 400 with field-level detail."

"But annotation validation only covers *shape*. Business validation lives in the service — 'is this plan an upgrade?', 'does this shop have credit?', 'is this code still valid?'. Those need database state, so they can't be annotations, and they must be in the service rather than the controller so that no future caller can route around them. That's the same reasoning behind putting the retailer-only check inside `RewardPointsService` rather than on the endpoint."

Q What's the difference between `@RequestBody String` and `@RequestBody SomeDto`? Why does it matter here?

A "`SomeDto` lets Jackson deserialise. `String` gives you the raw body bytes as sent. In the Razorpay webhook I take the raw `String` deliberately, because HMAC is computed over the exact bytes Razorpay signed. Deserialising and re-serialising would reorder keys or normalise whitespace, and the signature would no longer match. Any signature-verified webhook has to be handled at the raw-body level."

26 · Question bank — Security & authentication

Q Explain your authentication flow end to end.

A Walk it: "User posts credentials → `AuthService` verifies the BCrypt hash → issues a short-lived HS256 JWT (subject = user UUID, email claim) plus an opaque refresh token whose **SHA-256 hash** is stored. The Next.js server action writes both into HttpOnly cookies. On subsequent requests the browser sends no token at all — it can't, the cookie is HttpOnly — and the BFF route reads it server-side and attaches the Authorization header. `JwtAuthFilter` verifies the signature, loads the user, and populates the `SecurityContext`. When the access token expires, `middleware.ts` refreshes it before render, because Server Components aren't allowed to mutate cookies."

Q Why store refresh tokens hashed?

A "Same reason you hash passwords. A refresh token is a bearer credential valid for seven days — anyone holding it can mint access tokens. If the database leaks and I stored them in plaintext, the attacker gets a week of live sessions for every user. Storing SHA-256 means the dump is useless: verification hashes the presented token and compares."

"Why SHA-256 rather than BCrypt? Refresh tokens are high-entropy random values, not user-chosen passwords, so there's no dictionary to attack and no need for the deliberate slowness. BCrypt on every token refresh would be a pointless CPU cost on a hot path."

"Migration `V2` truncated the table when this shipped, so every user was logged out exactly once. That was the right call — silently leaving plaintext rows would have kept the vulnerability alive."

Q Why JWT instead of server-side sessions?

A "Statelessness. Any API instance can verify any token with just the signing key — no session store, no sticky sessions, so horizontal scaling is free. It also means multiple client types (web, and eventually the Android app) authenticate identically."

"The trade-off is real and I'll name it: **you cannot revoke a JWT**. If a token is stolen it's valid until it expires. I mitigate that three ways: short access-token TTL, refresh tokens that *are* server-side state and therefore revocable, and HttpOnly cookies so a stolen token is much harder to obtain in the first place. If I needed true instant revocation I'd add a Redis denylist keyed on token ID — which reintroduces the state I was avoiding, so it's a conscious trade, not an oversight."

Q How do you prevent XSS from stealing a session?

A "Structurally, not defensively. The browser never holds a token. There is nothing in `localStorage`, nothing in `sessionStorage`, nothing in a JS-readable cookie. Client code calls `/bff/api/...`, and a Next.js route handler reads the HttpOnly cookie server-side and attaches the Authorization header."

"So even a full XSS — an attacker running arbitrary JavaScript on my origin — cannot exfiltrate a session. They can make authenticated requests *while the user is on the page*, which is bad, but they can't take the credential away and replay it later from their own machine. That's a meaningful reduction in blast radius."

"On top of that, a CSP with no `unsafe-eval` in production, a `connect-src` that doesn't even name the backend origin, and React's default escaping."

Q You disabled CSRF. Isn't that a vulnerability?

A "No, and the reasoning matters. CSRF exploits *ambient credentials* — the browser automatically attaching a cookie to a cross-site request. The Spring Boot API doesn't authenticate by cookie; it authenticates by an `Authorization` header that a cross-site attacker cannot cause the browser to send. So there's nothing for CSRF to exploit at that layer."

"The cookies do exist, but they live between the browser and the *Next.js* server. That layer is protected three ways: mutating operations go through Next server actions, which require a same-origin request; the cookies are `SameSite=Lax`; and the CSP sets `form-action 'self'`. So CSRF is handled where cookies are actually used, and disabled where they aren't."

Q How does role-based access control work, and what's beyond roles?

A "Three layers, and I'd argue the second and third are the interesting ones."

- **Roles** — five (`ADMIN`, `DISTRIBUTOR`, `RETAILER`, `PAINTER`, `CUSTOMER`) plus a `GUEST` principal, enforced by URL rules and `@PreAuthorize`.
- **Feature grants** — a distributor switches individual product pages on or off per shop. Rather than a boolean column per page (a schema change every time a page ships), it's an `AppFeature` enum stored as rows and enforced by an interceptor. Crucially, dashboard/account/subscription are deliberately *not* grantable — revoking them would create an account that can't fix its own billing.
- **Ownership** — `ProjectAccessPolicy` answers "may this principal see this project?" separately from "who pays for it?" (`ProjectBillingResolver`). A redeemed customer's project is owned by the customer and billed to the shop. Conflating those is how you leak one customer's room photo to another.

Q Walk me through the Google OAuth2 flow.

A "Standard authorization-code flow via `spring-boot-starter-oauth2-client`. Google redirects back, Spring exchanges the code for tokens, and `CustomOAuth2UserService` upserts the user — matching on email so a user who registered locally and later signs in with Google lands on the same account rather than a duplicate."

"The non-standard part is the last hop. `OAuth2AuthenticationSuccessHandler` doesn't put JWTs in the redirect URL, because URLs end up in browser history, server logs and `Referer` headers. Instead it issues a **one-time exchange code** — a short-lived database row — and redirects with that. The frontend then POSTs it to `/api/auth/oauth2/exchange` and gets real tokens back in the response body. One extra round trip, and no credential ever appears in a URL."

Q How do you rate limit, and why fail open?

A "Per-IP fixed window in Redis — INCR plus EXPIRE — applied to every sensitive endpoint: register, login, refresh, password reset, OTP send and confirm, access-code redeem, subscribe/verify, image upload, kiosk order. Over-limit returns 429 with `Retry-After`."

"It fails **open**: if Redis is unreachable, requests pass. That looks wrong until you ask what the control is for. A rate limiter protects *availability* and cost. If it fails closed, a Redis blip means nobody can log in — I've converted a degraded dependency into a total outage, and the attack I was defending against wasn't even happening."

"Contrast that with webhook signature verification, which fails **closed**: that's an *integrity* control on a public endpoint, and letting unsigned events through would let anyone grant themselves a paid plan. Same question, opposite answers, because what's being protected is different. That's the framework I'd apply to any 'what if the dependency is down' decision."

Q How do you know the client IP is real?

A "I don't trust `X-Forwarded-For` naively, because it's client-supplied and anyone can rotate a fake IP per request to bypass every limit. `ClientIps` counts trusted proxy hops **from the right** — configured by `app.rate-limit.trusted-proxy-hops`, default 1 — so only the hop my own proxy appended is trusted, not whatever the client prepended."

"The frontend cooperates: `middleware.ts` *overwrites* the header with its own derivation before rewriting `/api/auth/*` to the backend, and deletes `x-real-ip`. That's necessary because Next's rewrite proxy forwards `X-Forwarded-For` exactly as the client sent it. And there's `RATE_LIMIT_TRUST_FORWARDED=false` for deployments where the backend port is directly reachable — in that case the header should not be believed at all."

Q What OWASP Top 10 items did you actively address?

Risk	Mitigation in this project
Broken access control	Three layers: roles, feature grants, per-resource ownership policy. `ImageControllerKeyGuardTest` proves one customer can't read another's photo.
Cryptographic failures	BCrypt passwords, SHA-256 refresh tokens, HS256 JWTs with a fail-fast secret, HSTS, TLS everywhere.
Injection	JPA parameter binding and the Criteria API — no string-concatenated SQL anywhere. React escapes by default.
Insecure design	Reserve-then-commit ledger, bounded queues, bulkheads, fail-open/closed reasoned per control.
Security misconfiguration	Swagger off in production, actuator minimally exposed, CORS never `*`, secrets with no defaults, boot-time config diagnostics.
Identification & auth failures	Rate-limited login, admin 2FA, hashed refresh tokens, enumeration-resistant password reset.
SSRF	Next image optimiser restricted to an explicit host allow-list — a blanket `**` would make it an open proxy.
Logging & monitoring	Immutable audit log, Prometheus metrics, production log level pinned at INFO so PII isn't emitted.

Q How would you handle a security incident — say the JWT secret leaked?

A "Rotate `JWT_SECRET` and redeploy. That invalidates every access *and* refresh token simultaneously — every user and guest session is signed out and must log in again. It's documented in `DEPLOYMENT.md` precisely so nobody discovers that consequence during the incident."

"Then: check the audit log for anything anomalous during the exposure window, force password resets if I believe accounts were touched, and work out how the secret leaked — was it committed, logged, or shared? The last one is the important step, because rotating without fixing the exposure path just resets the clock."

27 · Question bank — Database & JPA

Q Why PostgreSQL?

A "Relational data with real integrity requirements — a subscription references a user, a region references a project, a points transaction references a lot. Foreign keys and transactions are the point. Postgres specifically for strong constraint support (I use `CHECK` constraints reconciled against Java enums), JSON columns where I need semi-structured data, mature indexing, and because it's the boring, well-understood choice for a system where correctness of money matters."

"A document store would have made the hierarchy queries — 'all shops under this distributor, with their brand assignments and current plan' — into application-side joins, which is where consistency bugs live."

Q Why Flyway? Why not `ddl-auto=update`?

A "`ddl-auto=update` is convenient and dangerous. It infers a diff at startup, it never drops anything, it can't express a data migration, it isn't reviewable in a pull request, and it silently drifts between environments. You find out in production."

"Flyway makes the schema a versioned artefact — 37 numbered SQL files, each reviewed like code. Production runs `ddl-auto=validate`, so Hibernate *checks* that entities and schema agree and refuses to boot if they don't. Change an entity and forget the migration, and the app tells you at startup instead of failing on a query at 3 a.m."

"Adopting it mid-project needed care: `baseline-on-migrate=true` stamps existing non-empty databases as already at V1 and applies only V2 onward, so live databases created by the old `update` mode were adopted with zero manual steps. And there's a `MigrationsCoverEntitiesTest` so the discipline is machine-enforced."

Q How do you handle N+1 query problems?

A "First, notice them — `show-sql` in development, or Hibernate statistics, and count queries per request. Then the standard tools: `JOIN FETCH` in a JPQL query, `@EntityGraph` on the repository method, or batch fetching."

"But the structural answer is that my read-heavy endpoints don't return entities at all. The catalogue returns `ShadeSummaryResponse` projections, not `Shade` entities, so there are no lazy associations to trigger in the first place. A projection can't N+1. Combined with Redis caching on that endpoint, the hottest path in the product barely touches the database."

Q Explain the JPA Specifications you used.

A "The catalogue filters by brand, family, temperature, tonality and free-text search, in any combination. That's 2^5 permutations. Writing a repository method per combination is unmaintainable; concatenating JPQL strings invites injection and typos."

"Specifications wrap the JPA Criteria API into composable predicates. Each filter is a `Specification<Shade>` that returns null when the parameter is absent, and they compose with `.and()`. One line builds any combination, it's type-safe against the metamodel, and it's parameter-bound so injection is impossible."

```
Specification<Shade> spec = Specification
        .where(hasBrand(brand)).and(hasFamily(family))
        .and(hasTemperature(temp)).and(matchesSearch(q));
Page<Shade> page = shadeRepository.findAll(spec, pageable);
```

"The trade-off is readability — Criteria code is verbose and harder to eyeball than SQL. For genuinely dynamic filtering it's worth it; for a fixed complex query I'd write JPQL or native SQL."

Q How do you prevent race conditions on quota?

A "The naive flow — check remaining quota, do the work, decrement — has a TOCTOU race. Two concurrent requests both read '1 remaining', both pass the check, and the shop gets two projects for one credit."

"I use **reserve-then-commit**. `reserveAiUsage()` decrements the quota inside a transaction *before* any upstream call, so the second request sees zero and is rejected. If the pipeline succeeds, `incrementAiUsage()` commits the hold; if it fails, the hold is refunded. That's a two-phase pattern, and it's exactly why `ProjectHoldAccountingTest` exists."

"For the underlying row I'd reach for optimistic locking (`@Version`) or a `SELECT … FOR UPDATE` depending on contention; the important part is that the decrement and the check are one atomic operation rather than two."

Q Optimistic vs pessimistic locking — which and when?

A "Optimistic (`@Version`) assumes conflicts are rare: read, modify, and on write check the version hasn't moved; if it has, throw and let the caller retry. Cheap, no locks held, scales well. Right for most of this system — two people rarely edit the same project simultaneously."

"Pessimistic (`SELECT … FOR UPDATE`) takes the lock up front. Right where conflicts are likely and a retry is expensive or user-visible — a points balance being spent concurrently, for example, where an optimistic failure would surface as a confusing 'please try again' on a payment."

Q How would you scale the database?

A "In order of cost. (1) Indexes and query tuning — `EXPLAIN ANALYZE` the real slow queries. (2) Caching, which I already do for the catalogue. (3) Read replicas — the read/write ratio here is heavily skewed, and the catalogue is entirely read-only, so routing reads to replicas gets a long way. (4) Connection pooling in front of Postgres with PgBouncer, because Hikari's 20 connections per instance multiplies by instance count. (5) Partitioning the append-only tables — `audit_log` and the points transaction ledger are time-series shaped and partition cleanly by month. (6) Only then, sharding — which I'd avoid as long as possible because it breaks cross-shard joins and transactions."

Q How do you model money?

A "`int` of paise, never a floating-point type. ₹999 is `99900`. Floats can't represent 0.1 exactly, and in a billing system accumulated rounding error becomes a support ticket you can't answer. Paise also matches Razorpay's own API, so there's no conversion at the boundary — one fewer place to get it wrong."

"Sentinel `-1` means custom pricing (Enterprise), and `Integer.MAX_VALUE` means unlimited. The second one is a genuine trap: multiply `MAX_VALUE` by a subscription quantity and you overflow to a negative number, which reads as 'no quota' and locks out your biggest customer. There's a test named `UnlimitedPlanQuotaOverflowTest` for exactly that."

Q Your timestamps are zone-naive. Defend that.

A "I won't fully defend it — it's the thing I'd change first. The schema stores `LocalDateTime`, so every expiry is interpreted in the JVM's default zone. I pinned `TZ=Asia/Kolkata` in the Dockerfile and repeated it in compose, and documented it in the production checklist, because moving the JVM to a UTC host would silently shift every share link, subscription period and access-code expiry by five and a half hours."

"It's correct for a single-country product and it was cheap. The right design is `Instant` stored as `timestamptz`, formatted per-user at the edge. Knowing the difference — and knowing that the mitigation is a deployment constraint rather than a fix — is the honest answer."

Q Why are there 43 entities? Isn't that over-modelled?

A "They're spread across 17 feature modules, so it's roughly two to three per module — auth has six, billing has eight, project has two. The billing ones look numerous until you see they're an accounting system: `Subscription` and `SubscriptionPayment` separate the mandate from the individual charges; `RewardPointsLot` and `RewardPointsTransaction` separate the FIFO expiry buckets from the immutable ledger; `ProcessedWebhookEvent` is a pure idempotency marker."

"That last split is deliberate double-entry-ish thinking: lots hold balance with expiry, transactions record what happened and are never mutated. If you collapse them you can't answer 'why is my balance what it is' — which in a paid product is a question you must be able to answer."

28 · Question bank — Razorpay & payments

Q Walk me through your Razorpay integration.

A Give the shape first, then depth: "Three flows. Monthly plans use Razorpay **Subscriptions**. Buying points and the ₹99 kiosk sale use **Orders**. Both verify a Checkout signature server-side before granting anything. On top of that a **webhook** receiver handles the subscription lifecycle and refunds, because Checkout verification only covers the first payment — renewals, failures, cancellations and refunds all arrive by webhook."

Then walk the sequence diagram from section 11.3.

Q How do you verify a payment is real?

A "Two different signatures, and it's worth being precise about which is which."

"**Checkout signature** — proves a specific payment happened. For a subscription it's `HMAC_SHA256(payment_id + "|" + subscription_id, KEY_SECRET)`; for an order it's `HMAC_SHA256(order_id + "|" + payment_id, KEY_SECRET)`. Compared server-side against the `razorpay_signature` the browser handed back. Without this, a hostile client could just POST a made-up payment ID to my verify endpoint and get a Business plan."

"**Webhook signature** — proves the event came from Razorpay, computed over the raw request body with a *separate* webhook secret. Two secrets for two purposes."

Q What if the webhook secret isn't configured?

A "Reject every webhook. Fail closed."

"The tempting answer is 'log a warning and process anyway so we don't lose events.' That would be a critical vulnerability: the webhook endpoint is `permitAll` by necessity — Razorpay can't authenticate with a JWT — so anyone on the internet could POST a forged `subscription.activated` and grant themselves a plan, or forge `payment.captured` to reset quota. Losing events is recoverable through the dashboard; granting free plans to attackers is not."

Q How do you handle duplicate webhooks?

A "Razorpay retries with exponential backoff whenever it doesn't get a 2xx, so duplicates are guaranteed, not hypothetical. I key idempotency on `X-Razorpay-Event-Id`: if a `ProcessedWebhookEvent` row exists, skip; otherwise insert the marker and process."

"Three properties make it actually correct rather than approximately correct:"

- "The marker insert shares the handler's transaction. If processing fails halfway, the marker rolls back with it — so the retry is processed cleanly rather than skipped as already-done. Marking outside the transaction would turn a transient failure into permanent data loss."
- "A duplicate racing insert dies on the primary key. Two concurrent deliveries: one wins, the other gets a constraint violation, returns 5xx, and the retry finds the marker and skips. **The database is the concurrency control** — no application-level locking, no distributed lock."
- "A missing event ID is logged loudly rather than silently trusted."

Q Why return 500 on a processing error instead of 200?

A "Because with webhooks, **200 means 'I have durably handled this'**. It tells Razorpay to stop retrying and forget the event."

"If my database was briefly unavailable during an activation and I returned 200, I'd have permanently lost a real paying customer's plan activation with no way to replay it. Returning 500 enlists Razorpay's own exponential-backoff retry as my recovery mechanism. Combined with the idempotency marker, retries are completely safe — at-least-once delivery, exactly-once processing."

"I do distinguish failure types: a bad signature returns **401**, because retrying a forged request is pointless. And the raw exception message is never echoed back — that's an unauthenticated endpoint and the message could leak schema detail."

Q Why verify at Checkout AND handle webhooks? Isn't one enough?

A "They solve different problems and you need both."

"Checkout verification is about **immediacy**. The buyer finishes paying and the plan is live before the modal closes. If I waited for the webhook, they'd stare at a 'pending' screen for anywhere between two seconds and two minutes and then email support."

"The webhook is about **durability**. It covers everything the browser can't report: the renewal in month two, a failed card in month five, a cancellation from the Razorpay dashboard, a refund, or the buyer closing the tab between paying and the callback firing. Both paths converge on the same idempotent `activateSubscription()`, so it doesn't matter which arrives first."

Q Which webhook events do you handle and why those?

A Run the table from section 11.7. Then add the two details that show you thought about it: "`subscription.charged` is the *authoritative* renewal event because it carries the real `current_end`, so period dates stay exact. `payment.captured` is a fallback if that isn't delivered — it passes 0 so the service falls back to +30 days, and it will never *shrink* a period end that the authoritative event already set. And unknown events are logged at debug rather than thrown, because throwing on an event type Razorpay adds next year would make it retry forever."

Q What happens if a payment fails mid-subscription?

A "Razorpay retries the charge on its own schedule — that's the value of using Subscriptions rather than rolling my own recurring billing. `payment.failed` arrives and I log it with the reason, so a spike is visible without digging through the dashboard. If Razorpay ultimately gives up, `subscription.halted` arrives and the subscription moves to a terminal state."

"Separately, a daily `@Scheduled` sweep hard-expires paid subscriptions whose period has lapsed — but only after a **3-day grace window**. Without that grace, a webhook delayed by an hour would cut off a customer who has genuinely paid. Webhook is the fast path, the job is the backstop, and the grace window is the apology for the gap between them."

Q How do you handle plan upgrades without double-charging?

A "This was a real bug. Originally, upgrading mid-cycle billed the new plan immediately *and* superseded the old row on activation — so the customer paid twice and lost the days they'd already paid for."

"The fix is `scheduledStartFor(userId)`: compute when the current period ends and pass it to Razorpay as `start_at`. The new subscription row carries that start date from the moment it's created, so entitlement queries can distinguish 'bought, starts later' from 'in force now'."

"There's also a rule that only *upgrades* may happen in place — `isUpgradeFrom()` compares enum ordinals, so declaration order is the tier ladder. Downgrades must cancel first, so nobody accidentally double-pays for a lateral move. And a separate fix: a subscription already scheduled to end no longer blocks a new purchase, which was the cancellation trap where cancelling locked you out of resubscribing until the period expired."

Q How do you handle refunds?

A "`refund.created` and `refund.processed` both route to `walletService.reverseKioskPayment(paymentId, amount)`. The reasoning is specific to the kiosk flow: a walk-in pays ₹99 and the shop earns 30 reward points. If that payment is later refunded or charged back, those points must stop counting toward the shop's balance — otherwise a pay-then-chargeback cycle mints free points indefinitely. Reversal on refund closes that loop."

Q The Key ID goes to the browser. Isn't that a leak?

A "No — it's public by design. Razorpay Checkout runs in the browser and needs the Key ID to identify the merchant account. It's in the client bundle and that's expected."

"The **Key Secret** is what proves a payment is real, and it exists only in the Spring Boot process's environment. It must never appear in the Next.js repository or in any `NEXT_PUBLIC_*` variable — anything with that prefix is inlined into the JavaScript bundle at build time and is therefore world-readable. That distinction — which half of the key pair is public — is the thing to be precise about."

Q Why points instead of a cash wallet?

A "A regulatory constraint expressed as an architectural one. If a shop could convert a balance to a bank transfer, every kiosk sale would become 'a payment collected on the shop's behalf' — which in Indian payments regulation makes you a payment aggregator, a licensed activity I'm not licensed for."

"So the kiosk price is flat and platform-set at ₹99, the shop takes no cash share, and it's rewarded in closed-loop points instead. There is deliberately **no payout method** on `RewardPointsService`, and a comment on the class says there must not be one. Earned and bought points are also indistinguishable once credited — same expiry clock, same prices — so the ledger never has to answer 'which kind of point was that', which is what would make a refund or an expiry ambiguous."

Q How would you test payments without spending money?

A "Three levels. Razorpay **test mode** gives you `rzp_test_` keys, test cards and test UPI — no KYC needed and no real money moves. For automated tests, the `RazorpayClient` is a Spring bean, so it's mocked; `BillingWebhookIntegrationTest` constructs signed payloads and asserts on signature rejection, idempotent replay and each event type. And for local development, missing credentials produce a client with empty keys that fails at call time with a readable message rather than crashing at startup — so I can work on the catalogue without payment credentials."

Q What's the biggest risk in your payment code?

A Answer honestly — this is a test of whether you've actually thought about it: "The app doesn't verify that the amount configured on the Razorpay dashboard plan equals the price in `Plan.java`. If someone creates the Professional plan at ₹2,000 while my code says ₹2,499, the pricing page advertises one number and the gateway charges another. That's a chargeback and a gateway-review problem, and it's a silent failure — nothing errors."

"The fix is a startup check that fetches each plan via the API and asserts the amount matches, failing the boot if not. It's not implemented yet and it's the first thing on my list. The coupling is documented in `RAZORPAY_SETUP.md` with a warning box, because Razorpay plan amounts **cannot be edited after creation** — so if GST ever changes from 0 to 18, the plans have to be recreated, not updated."

29 · Question bank — AI/ML integration

Q You say "AI project" — but you didn't train a model. Is it really an AI project?

A Meet this head-on; being defensive here is far worse than the question:

"I didn't train a model, and I'd say so on my resume. What I did is **AI systems engineering**, which is the job most companies are actually hiring for: choosing the right model for each task, designing prompts under a hard cost ceiling, orchestrating four models across two providers, and — the part that's genuinely hard — building the deterministic machinery around probabilistic components so the product is reliable even when the model isn't."

"Concretely: the white-salvage rule exists because the model disobeys its instructions in a specific, predictable way. The ΔE matcher exists because an LLM will hallucinate product codes, so I let it propose colours and let colour science map them to real products. The reserve-then-commit ledger exists because model calls fail and the customer shouldn't pay for it. Training a model is one skill; making a model's output safe to build a business on is a different one, and it's the one this project demonstrates."

Q Why Claude for some tasks and Replicate for others?

A "They're different capabilities. Claude is a language-and-vision model: it's excellent at classification, structured reasoning about an image, and generating text — so it does the INDOOR/OUTDOOR/INVALID triage, the colour recommendations, the shade enrichment and the support agent."

"Replicate is a hosting platform for image models. The segmentation work needs image *output*, not text — an edited image or a mask — which is a different model class entirely. SAM 2 is Meta's segmentation model and Replicate is the simplest way to call it without running GPU infrastructure."

"Within Claude I also pick by cost: **Haiku** for classification, which is a one-word answer where a bigger model buys nothing; **Sonnet** for recommendations, which needs actual aesthetic judgement. Using Sonnet for classification would be roughly ten times the cost for identical accuracy."

Q How did you control AI costs?

A Five distinct levers, and I'd list them as such:

- **Resize before classify.** Downsampling to 1024 px JPEG q85 before sending to Claude Vision cuts input tokens ~10× with no accuracy loss for a three-way classification. That single Thumbnailator call is the difference between ₹0.30 and ₹3 per upload.
- `max_tokens = 10` on classification. A hard ceiling on output cost — the model literally cannot run away.
- **Model selection by task.** Haiku where Haiku suffices.
- **Architectural.** The big one: AI runs once per project, not once per colour change. That's the ₹5 → ₹0 move.
- **Bounded demand.** Quota enforcement per plan, per-IP rate limiting on the endpoint that isn't quota-charged, and a queue depth cap of 500 so a burst can't produce unbounded spend.

Q How do you handle non-deterministic model output?

A "You assume it will be wrong in plausible ways and you write the deterministic layer to tolerate that. Four examples from this codebase:"

- "**Constrained output format.** The classifier is told 'exactly one word only, no explanation' and anything unexpected falls through the `switch` to `null`, which means INVALID. Unknown input is a defined outcome, not a crash."
- "**Defensive parsing.** `stripCodeFences()` exists because Claude sometimes wraps JSON in ```` ```json ```` despite being told not to. Handling that in one shared place beats debugging it in five call sites."
- "**Salvage rules.** The mask model sometimes leaves an accent wall white instead of green — because that wall is already white in the cleaned photo. Rather than dropping those pixels and returning two masks where the UI expects three, a large near-white area is adopted as the accent when no green exists."
- "**Sanity thresholds.** Each mask is checked with `countForeground()` against a size threshold. A near-empty mask means the model couldn't find that surface, so the category is skipped rather than persisting noise as a region."

Q Walk me through your prompt engineering.

A "Two very different prompts, illustrating two different problems."

"**Classification** is about constraining output: enumerate the exact allowed answers, define the edge case explicitly ('INVALID means it is NOT a house or room image — e.g. selfie, food, nature, animal, document, car'), and forbid explanation. Then cap `max_tokens` at 10 so verbosity is impossible even if the instruction is ignored."

"**Masking** is about spatial precision, and the key insight took several iterations: I don't ask the model to *draw a segmentation map*, I ask it to **repaint the actual surfaces** in flat colours. Painting onto real geometry keeps the output pixel-aligned to the canvas; drawing an abstract map drifts. I also enumerate what must stay BLACK — sky, ground, stone, windows, fixtures, and specifically door panels as dark brown and metal railings as charcoal grey — because those are the surfaces the model most often mistakes for paintable wall."

"And I chose **high-chroma RGB** as the encoding rather than greyscale levels or subtle hues, because saturated primaries survive the lossy JPEG round-trip through the model's output encoder. That's a prompt decision driven by an image-compression constraint."

Q Why do you match colours in CIELAB instead of RGB?

A "Because RGB distance doesn't correspond to perceived difference. A numeric distance of 30 in the greens is invisible; the same 30 in the blues is obvious. If you pick the nearest paint chip by RGB distance you'll frequently pick one that looks wrong to a human, which in a colour-matching product is the only thing that matters."

"CIELAB is designed to be perceptually uniform — equal numeric distances are roughly equal perceived differences. The conversion is sRGB → linearise → XYZ (D65 matrix) → L*a*b*, then ΔE is Euclidean distance in that space. Under $\Delta E < 1$ humans can't tell two colours apart at all."

"I use **CIE76**, the 1976 formula. CIEDE2000 is more accurate, especially for saturated colours and near-neutrals, at the cost of a much more complex formula with hue-rotation and chroma-compensation terms. For 'find the closest chip out of 8,000' the ranking rarely differs, so CIE76 was the right trade. But I know the difference and I'd switch if matching quality ever became a complaint."

Q Why snap AI-suggested colours to catalogue shades?

A "Because you can't buy a hex code. If I let the model name shades directly, it will hallucinate product codes — and a hallucinated Asian Paints code handed to a customer in a shop is worse than no suggestion at all. So the model proposes *colours* and deterministic colour science maps them to *products*. The LLM does taste, the maths does correctness."

"The response also carries the ΔE alongside each match, so the UI can show how close the real paint actually is to the AI's idea. That's honest — the customer sees when the catalogue doesn't have a close match rather than being told a distant shade is 'the recommendation'."

Q What happens when an AI service is down?

A "It depends on whether the feature is load-bearing, and I chose deliberately per caller — that's why `ClaudeService` exposes two methods with different failure semantics."

- "**Classification fails hard.** `ExternalServiceException` → 503 with a retry message. Storing an unclassified image would corrupt every downstream assumption about it."
- "**Support agent fails soft.** `complete()` returns `Optional.empty()` and the widget degrades to a contact form. Nobody's blocked from buying paint because a chatbot is down."
- "**Segmentation fails into a state machine.** The project moves to `FAILED`, the held credit is refunded, and the UI offers a retry — the customer is not charged for my outage."
- "**Cleaning hints fail soft** — they're an enhancement inside the cleaner."

Q How do you test AI-dependent code?

A "Mock the client at the Spring bean boundary. `ClaudeService` and the Replicate segmenter are injected, so tests substitute deterministic fakes — no network, no keys, no cost, no flakiness. `StubAiPipelineTest` covers the stub mode itself."

"There's also the stub pipeline in the product, which is more interesting than a test double: `HUEVISTA_TESTING_STUB_AI_ENABLED` replaces the two paid Replicate calls with a locally-drawn mask — three vertical stripes in RED, GREEN and BLUE — so every downstream feature has real three-category data to work on. Crucially, **billing is untouched**: a stub run charges credits exactly like a real one, so plan limits stay testable. That let me click-test the entire product end to end for free without the test mode silently making the billing path untested."

30 · Question bank — Frontend, React & Next.js

Q Why Next.js instead of plain React?

A "Three things I actually needed. **Server-side rendering** for the marketing pages — this is a product sold to shops that will find it by search, so static, fast, crawlable pages matter commercially. **The server runtime**, which is what makes the BFF pattern possible: middleware and route handlers let me hold tokens server-side, which a pure SPA fundamentally cannot do. And **routing, code-splitting and image optimisation** without assembling them myself."

"A plain React SPA would have forced tokens into the browser, and I'd have had to build SSR separately for SEO."

Q Explain Server Components vs Client Components.

A "Server Components render on the server, never ship their code to the browser, and can access server resources — cookies, the database, secrets — directly. Client Components carry `"use client"`, ship to the browser, and can use state, effects and event handlers."

"In my app the boundary follows interactivity. Marketing pages and app page shells are Server Components — smaller bundles, faster first paint. The studio is a Client Component because it needs WebGL, canvas and continuous interaction."

"The constraint that bit me: **a Server Component must not mutate cookies during render.** Token refresh was originally happening lazily inside a Server Component and every user with an expired token hit *'Cookies can only be modified in a Server Action or Route Handler.'* The fix was to move refresh into `middleware.ts`, which runs before render where cookies are writable, and make `getAccessToken()` read-only by contract. The framework constraint told me where the work belonged."

Q What is the BFF pattern and why use it?

A "Backend-for-Frontend: a server-side layer that sits between the browser and the API, owned by the frontend team, shaped to the frontend's needs."

"Here it exists primarily for **security**. The browser calls `/bff/api/...` with no credentials at all — it can't send any, the cookie is HttpOnly. A Next route handler reads the cookie server-side, attaches the Authorization header, and forwards. The result is that no token is ever reachable from JavaScript, so XSS can't exfiltrate a session."

"It also gives me a place to enforce a **path allow-list** so the proxy isn't an open relay, to filter headers in both directions so upstream banners don't leak, to forward the real client IP so the backend's per-IP limiters bucket correctly, and to swap in a demo mode that answers from fixtures with no backend at all."

"The cost is an extra network hop and a Node process in the path. Worth it."

Q How does your middleware work?

A "Two jobs. **Route gating**: eleven protected prefixes redirect to `/sign-in` without a session, three guest-only paths bounce signed-in users home, and unpublished editorial routes rewrite to a real 404 — done in middleware rather than `notFound()` inside the page, because the root `loading.tsx` opens a Suspense boundary and the shell is already flushed by the time a page-level `notFound()` throws, so a crawler would see the 404 page under a 200 status and keep the URL indexed."

"**Token refresh**: if the access cookie is gone but the refresh cookie is present, POST to refresh and write the new cookie — in middleware, because that's where cookies are writable."

"One wrinkle: Next requires `config.matcher` to be a static literal, so the route list necessarily exists twice in the file. A route missing from either copy silently stops refreshing and starts bouncing people to sign-in. Rather than trust a comment, there's a test that holds both lists against the pages that actually exist under `(app)`. When the framework forces duplication, make a test the source of truth."

Q How do you handle loading and error states?

A "Next's file conventions do most of it: `loading.tsx` defines the Suspense fallback for a route segment, `error.tsx` is an error boundary, and there are separate ones at the root and inside the `(app)` group so an error in the authenticated shell doesn't blow away the marketing site."

"Streaming SSR means the shell renders immediately and data streams in, so users see structure rather than a spinner on a blank page. For the segmentation poll specifically there's a dedicated `segmentation-polling.ts` module with backoff, and a `pipeline-bar` component that shows which stage the job is in — because a 30-second wait with no feedback feels broken even when it's working."

Q Why plain CSS instead of Tailwind or styled-components?

A "A design-token system with CSS custom properties gives me theming — light and dark — with zero JavaScript and zero runtime cost. CSS-in-JS adds a runtime and complicates Server Components. Tailwind is fine, but it puts the design system in class names rather than in tokens, and I wanted the token layer to be first-class because the whole product is about colour."

"The honest cost: more verbose authoring, and I have to maintain naming discipline myself rather than having a framework enforce it. For a solo project where I control every file, that was an acceptable trade."

Q How do you handle forms and validation?

A "Next **server actions** for mutations — which means the CSRF protection comes free from same-origin enforcement, and no client-side fetch code to write. Validation happens twice on purpose: client-side for immediate feedback, server-side because client validation is a UX feature, not a security control. The backend validates independently with Bean Validation, so a crafted request that bypasses the browser entirely still fails."

Q How would you improve frontend performance?

A "Measure first — Lighthouse and real Core Web Vitals, not intuition. Then the specific things I know about this app: the studio route is the heaviest because it pulls in the WebGL engine and canvas code, so that's the first candidate for dynamic import behind an interaction. Masks are PNGs from S3 and could be served as WebP with proper cache headers via CloudFront. GSAP and Motion are both in the bundle and there's probably overlap I could consolidate."

"Structurally it's already in decent shape: marketing pages are static RSC, code splitting is route-based, fonts are preconnected with `font-display: swap`, and app pages stream."

Q How do you test the frontend?

A "Vitest with Testing Library on jsdom — 40 suites. The philosophy is testing behaviour rather than implementation: query by role and label the way a user would, not by class name."

"The suites that earn their keep aren't the component ones, though — they're the **guard** tests: `app-nav-access.test.tsx` proves the nav only shows features the shop has been granted, `protected-routes.test.ts` holds both copies of the middleware route list against reality, and there's a test that holds the BFF path allow-list against every path the API client actually calls. Those catch the class of bug that's invisible in review — a list that drifted out of sync with another list."

"What's missing: end-to-end tests. Playwright against a running stack would cover the flows that span both repositories — redeem a code, upload, segment, recolour, share. That's the biggest gap in my testing story and I'd say so."

31 · Question bank — Graphics & WebGL

Q Explain how the recolouring actually works.

A "The naive version is `mix(photo, colour, mask)` — and it looks like a paint-bucket fill, because it throws away every shadow and every bit of texture. The wall becomes a flat rectangle."

"What I do instead is decompose the photograph into two layers. A blurred copy is the **form** layer — the large-scale light: eaves, reveals, the gradient of light falling across the wall. Photo minus blur is the **detail** layer — the high-frequency truth: plaster stipple, dirt, seams, micro-shadows. Then the paint is reassembled as `target × form + detail`."

"That's why it reads as real paint on a real wall: the surface's actual texture is carried onto whatever colour the user picks. A flat fill can't fake it, and synthetic grain looks synthetic."

Q Why WebGL instead of Canvas 2D?

A "Per-pixel work on the GPU in parallel versus on the CPU in a JavaScript loop. A 12-megapixel photo is 12 million pixels; doing the form/detail maths per pixel per frame in JavaScript won't hit 60 fps on a mid-range Android tablet, which is the actual device at a shop counter."

"I do ship a Canvas 2D fallback — 351 lines behind the same `RecolorEngine` interface — for devices without WebGL2. Lower fidelity, but it works. In this market a meaningful share of counters run older hardware, and 'no WebGL2' can't mean 'no product'."

Q How do you paint multiple walls at once?

A "This was a real bug first. The original renderer redrew from the source photograph for each region, so painting the second wall reverted the first — the classic 'switching tabs wipes the last colour'."

"The fix was architectural, not a shader tweak: upload the photo to the GPU **once**, then draw one base pass with `u_useMask = 0`, followed by one blended pass per region with that region's mask texture bound and `u_target` set to its colour. All regions composite in a single frame. The base pass forces alpha to 1 so an exported PNG never has see-through holes from a transparent source."

Q What's the "scene-light anchoring" mode?

A "It's the piece I'm proudest of, because it's a rendering capability unlocked by a decision two steps upstream in the pipeline."

"The clean-up step repaints every paintable surface a known fresh white — LRV around 85, so sRGB around 0.94. That means the photograph of those surfaces is no longer 'a picture of a wall'; it's a **measurement of the light falling on that wall**, level and colour cast both. So in anchored mode I divide the blurred photo by that known albedo, per channel, and recover the scene's illumination directly."

"The practical difference: an evening photo keeps its dim, warm evening light instead of snapping to flat noon daylight. The legacy mode normalises by the region's own mean luminance so the wall averages to the exact swatch colour — which is right when you want to show 'the colour in the can', and wrong when you want to show 'this colour, in this room, at this time of day'. Both are available, dialable per region."

Q How do you avoid hard edges at the mask boundary?

A "Two things. The masks are smooth-upscaled from the model's resolution to the canvas resolution with bilinear interpolation, so the edge is already anti-aliased rather than a binary step. Then `featherMaskInward` softens the boundary **inward only** — feathering outward would bleed paint onto the window frame, which is exactly the artefact that makes a render look fake. Inward keeps the paint strictly inside the wall and lets the soft edge do the blending."

"The feather radius is computed in mask-space pixels (`featherRadiusInMaskPx`), so the softness is visually consistent whether the mask was stored at 512 or 2048 pixels. And the mask's soft edge is the *only* place colour blends — everywhere else the fill is exact."

Q Why avoid branches in a fragment shader?

A "GPUs execute in lockstep across groups of threads. When threads in a group take different branches, both paths execute and the results are masked — so a branch costs you the sum of both sides rather than one of them, and it runs millions of times per frame."

"In the highlight/shadow handling I compose the two halves arithmetically instead: the dark path is `target × pow(min(form,1), …)` and the lift path is a rolled-off `max(form-1, 0)`. Each equals `u_target` where it's inactive, so summing them and subtracting `u_target` composes both per channel with no conditional at all."

Q Why is the highlight lift "rolled off"?

A "A hard clamp at 1.0 reads as blown-out white patches — you get a flat white blob wherever the wall is brightly lit, and it looks like a rendering error. So the lift is compressed with `lift = (lift / (lift + 0.7)) * 0.7`, which asymptotically approaches a ceiling instead of hitting one. Bright areas get brighter but never clip."

"The same reasoning drives the whole-image brighten control being a **gamma** lift (`input^(1/u_bright)`) rather than a linear gain: gamma keeps pure white exactly where it is, so a bright sky in an exterior photo doesn't clip when the user lifts the midtones. And the brighten is applied to the base photo and the painted regions alike, so paint sits in the same light rather than floating dark on a lifted photo."

Q How do you export what's on screen?

A "The WebGL canvas is read back to a PNG. The base pass forces alpha to 1 specifically so the export never has transparent holes where the source image had them. For colour boards there's a client-side PDF generator that composes multiple recoloured snapshots into a document — which is why the per-document image cap exists per tier: it's a browser-memory guard as much as a pricing lever, and it's why even Enterprise has a finite cap at 16."

32 · Question bank — System design & scale

Q Design this system for 10,000 shops. What changes?

A Structure the answer as: what already works, what breaks, what I'd change.

"**Already fine:** the API is stateless — JWTs, no server sessions, and the job queue is external — so it scales horizontally behind a load balancer with no changes. Postgres handles this volume comfortably. S3 and CloudFront are effectively unbounded."

"**Breaks first:** the segmentation pipeline, and it breaks on *cost and third-party rate limits* before it breaks on my own capacity. 10,000 shops × 15 projects a month is 150,000 Replicate runs. My queue depth cap of 500 would start shedding load during peaks."

"**Changes I'd make:** split segmentation workers into their own autoscaling pool consuming the same Redis queue — the interface is already a message boundary, so that's a deployment change not a rewrite. Add read replicas and route catalogue reads there. Put PgBouncer in front of Postgres because Hikari's 20 connections multiplies by instance count. Partition `audit_log` and the points ledger by month. And negotiate committed throughput with Replicate rather than relying on burst capacity."

Q How would you make the system multi-region?

A "I'd resist it until there's a real reason, because it's a large step. The current design is single-region ap-south-1 and the customers are all in India, so latency isn't a problem."

"If I needed it — say expanding to the Gulf — the pieces are: CloudFront already handles static and image delivery globally. Postgres would go to a primary with cross-region read replicas, accepting that writes stay in one region. Redis would need per-region instances, which is fine for caching and rate limiting but means the job queue is regional — so segmentation workers would be regional too. And the timezone assumption breaks immediately, which is the concrete reason the `LocalDateTime` decision is on my list to fix."

Q How do you ensure data consistency across services?

A "Today it's a monolith with a single database, so the answer is boring in a good way: ACID transactions. Reserve-and-commit, the webhook marker sharing the handler's transaction, the credit refund on failure — all of it is one database, one transaction boundary."

"The interesting cases are where I *can't* have a transaction, because the other party is a third-party API. Razorpay is the example: I can't roll back a charge because my database write failed. That's why webhook processing is idempotent and returns 5xx to trigger retries — the pattern is at-least-once delivery plus exactly-once processing, which is the standard answer when you can't have distributed transactions."

"If I extracted segmentation into a service, that's where I'd need the saga pattern or an outbox table for the billing side-effects. Right now the queue gives me at-least-once and the ledger operations are idempotent, which covers it."

Q What's your caching strategy?

A "Cache what's read often, changes rarely, and is expensive to compute. That's exactly the paint catalogue: ~8,000 shades, read on every catalogue page load, changed only when someone reseeds."

"Tiered TTLs by volatility — 60 minutes on paginated result sets, 6 hours on the near-static lookups like distinct family names and brand summaries. Redis-backed, and the whole cache manager is `@ConditionalOnProperty` so tests run with `spring.cache.type=none` against Spring's no-op cache and don't need a Redis instance."

"What I deliberately **don't** cache: anything user-specific, anything billing-related, anything with authorisation implications. A stale quota reading is a money bug, and a cached response that crosses a tenant boundary is a data breach. The rule is that caching is for shared, public, immutable-ish data."

"Invalidation is TTL-based rather than event-based, which is the right trade here — a shade that appears an hour late is invisible to users, and event-based invalidation is where cache bugs live."

Q Why Redis for the queue instead of Kafka or SQS?

A "Redis was already in the stack for caching and rate limiting, so the queue cost me zero new infrastructure. RPOPLPUSH gives atomic move-to-processing, which is all I need for at-least-once delivery with crash recovery."

"Kafka would give me durability guarantees, replay, partitioning and consumer groups. I need none of those: my jobs are small, order doesn't matter, and I have one consumer type. It would be a broker to operate, monitor and upgrade for benefits I can't currently use."

"I'd switch when I have multiple consumers of the same event, need to replay history, or need throughput Redis can't handle. Naming the trigger for switching is more useful than defending the choice forever."

Q How do you handle a slow third-party API?

A "Four layers, and I use all of them."

- "**Get it off the request thread.** Segmentation is async behind a queue, so the user gets a 202 immediately and polls."
- "**Bound concurrency.** Click-to-segment must be synchronous, so it's behind a `Semaphore(8)`. Past that, 503 immediately rather than blocking a ninth Tomcat worker."
- "**Timeouts everywhere.** Polling caps at 30 attempts × 2 s = 60 s, then fails cleanly. An unbounded wait is a resource leak with extra steps."
- "**Bound the backlog.** Queue depth cap of 500, then 503 with 'try again in a few minutes'. Rejecting work you can't do beats accepting work you can't finish."

"What I'd add: a proper circuit breaker — Resilience4j — so repeated failures stop attempting for a cooldown rather than each request discovering the outage independently at 60 seconds a time."

Q How do you monitor this in production?

A "Actuator health for orchestrators and load balancers — status-only, with liveness and readiness variants, used by both the Docker `HEALTHCHECK` and compose's `service_healthy` gating. Micrometer to Prometheus for JVM, HTTP latency and status codes, Hikari pool utilisation and cache stats."

"Alerts I'd set: 5xx rate, p95 latency, connection-pool exhaustion, heap pressure — and product-specific ones that matter more: segmentation failure rate, queue depth trending up, and webhook processing errors, because a webhook failure is silent money loss."

"Logs rotate locally at 10 × 10 MB and the checklist says ship them somewhere durable. Production runs at INFO, never DEBUG, because DEBUG emits user emails and S3 object keys."

"The gap I'd close: no distributed tracing and no error aggregation. Sentry or equivalent, plus OpenTelemetry spans across the frontend/backend/AI boundary, would be the next investment."

Q How do you deploy without downtime?

A "The pieces are in place: `server.shutdown=graceful` with a 25-second drain so in-flight requests finish; health endpoints so a load balancer knows when a new instance is actually ready; a stateless API so old and new instances can serve simultaneously; and Flyway migrations that run at startup."

"The discipline that makes it work is **backward-compatible migrations** — during a rolling deploy, old and new code run against the same schema. So: add columns nullable, never rename in one step (add, backfill, switch reads, drop later), and never drop a column the previous version still selects. That's a rule I have to follow manually; `ddl-auto=validate` catches drift but not incompatibility."

"Honestly: the README says production deploy isn't wired up. CI runs tests on every PR; CD is manual. That's the next piece of work and I'd say so rather than implying otherwise."

Q Where's the single point of failure?

A "PostgreSQL. It's single-instance, and if it's down the product is down — there's no read-only degraded mode. That's the honest answer and the first thing I'd fix with a managed instance with automated failover plus replicas."

"Redis is a softer dependency by design: the rate limiter fails open, the cache falls through to the database, and the queue has a direct-async fallback path. So a Redis outage degrades rather than kills — that asymmetry was deliberate."

"Third parties are single points for specific features but not for the product: Replicate down means no new segmentation but existing projects still recolour, share and export, because that's all client-side."

33 · Question bank — DevOps & testing

Q Walk me through your testing strategy.

A "479 tests across both repos, and the shape matters more than the count. The bulk is **integration** — `@SpringBootTest` plus MockMvc against H2, exercising the full request → security → service → database → response path with real Spring Security in place. That's deliberate: most of my bugs would be wiring and authorisation bugs, and those are invisible to unit tests with mocked security."

"**Unit tests** for pure logic where they're cheap and fast: mask pixel processing, access policy, IP derivation, quota arithmetic. **Contract tests** pin the API response shapes so a rename doesn't silently break the frontend. And a **schema test** asserts the migrations cover the entities."

"Every external service is mocked, so the suite is hermetic — no network, no keys, no cost, deterministic — which is what makes running it on every PR viable."

Q Do you practise TDD?

A Be honest and specific about when: "Not universally. For pure logic with a clear specification — the ΔE matcher, quota arithmetic, mask thresholds — yes, test first, because the test *is* the specification and it's faster than manual verification."

"For the AI pipeline, no. I didn't know what the model would return until I'd called it, so I explored first and wrote tests to lock in the behaviour once I understood it. Writing a test against imagined output would have tested my imagination."

"Where I'm strict is **regression**: every bug that reached me got a test before the fix. `UnlimitedPlanQuotaOverflowTest` exists because I hit that overflow. `ProjectHoldAccountingTest` exists because I got the refund accounting wrong once."

Q What's in your CI pipeline?

A "GitHub Actions on every PR: `./mvnw verify` for the backend with H2 in-memory so no external infrastructure is needed; `npm run lint`, `npm run typecheck` and `npm test` for the frontend. Dependabot keeps dependencies current — the history has merged dependency PRs."

"What's missing and what I'd add, in order: coverage reporting with a threshold gate, a container image build and vulnerability scan, secret scanning, and Playwright end-to-end tests against a running stack. Then actual CD with staged rollout."

Q Explain your Docker setup.

A "Multi-stage build: a Maven stage compiles and packages, a slim JRE stage runs it — so the shipped image doesn't contain Maven, the JDK or the source tree. Runs as a non-root user, pins `TZ=Asia/Kolkata`, and declares a `HEALTHCHECK` against `/actuator/health`."

"`docker-compose.yml` brings up a prod-like local stack — Postgres 16, Redis 7, and the backend — with `depends_on: condition: service_healthy` so the app doesn't start before its dependencies are genuinely ready rather than merely started. And `JWT_SECRET: ${JWT_SECRET:?...}` makes `docker compose up` fail fast with a helpful message rather than booting on a missing key. That fail-fast pattern appears at three layers — the property with no default, the compose check, and the deployment checklist — because it's the one setting where running with a bad value is worse than not running."

Q How do you manage environments?

A "Spring profiles. `dev` runs on H2 in-memory with Flyway disabled and Hibernate DDL generation — zero external infrastructure, which is why H2 is `runtime` scope rather than `test`. `test` is similar with everything external mocked. Production is PostgreSQL with Flyway owning the schema and `ddl-auto=validate`."

"Everything else is environment variables through `${VAR:default}` placeholders, documented in a 10 KB `.env.example` that explains what each one does rather than just listing names. Plus a Postman collection with separate local and staging environments."

34 · Question bank — Behavioural & product sense

Q **Tell me about a difficult bug you fixed.**

A Use the accent-wall salvage bug — it has the best structure. Symptom → investigation → root cause → fix → lesson:

"Some rooms came back with two masks instead of three, so the UI's 'highlight wall' control had nothing to bind to. It was intermittent, which made it look like a flaky API at first."

"I built an admin mask-viewer page to see the raw model output — that was the turning point, because until then I was debugging a black box. The model was following the instruction, just not as I'd assumed: it was told to paint accent walls green, but when a wall was *already white* it 'repainted' it white. And my pixel classifier was dropping white as unassigned."

"The fix was a salvage rule: when no usable green region exists, adopt a large near-white area as the accent. A genuine green always wins; white is only the fallback."

"The lesson generalises: with a probabilistic component upstream, the deterministic layer below has to tolerate *plausible disobedience*, not just correct output and clear errors. And building the observability tool was what made the bug tractable — I should have built the mask viewer first."

Q **Tell me about a time you changed your mind.**

A "The billing model. I originally built a prepaid rupee wallet plus per-item cash checkout — you'd buy 'an extra image' or 'an extra auto-mask' separately. Two problems. Commercially, a shop could buy the wrong one and still be unable to finish a run — a cleaned photo with no way to mask it, which is a support ticket and a refund. Structurally, a cash balance the shop could draw down implied a cash-out path, and that would have made every kiosk sale a collection on the shop's behalf — a licensed activity."

"So I replaced it with a single closed-loop point currency and made the *project* the billable unit — one thing, one price, all-or-nothing. You can see the whole reversal in the migrations: `V17__billing_wallet` then `V31__points_only_billing`. That's a lot of work thrown away, and it was the right call."

Q **How do you decide what to build next?**

A "By what blocks the next real user, not by what's most interesting to build. The phasing shows it: Phase 1 was the minimum that makes a shop able to sell — upload, segment, recolour, share, and take money. Everything else waited."

"The distributor hierarchy came in Phase 2 not because it was fun but because distribution in this market goes through distributors, so without it there's no channel. Painter accounts and ordering are Phase 3 because they're upsell, not activation. True 3D via Gaussian Splatting is Phase 5 and explicitly 'demand-conditional' — I wrote that word down deliberately so I wouldn't build it because it's exciting."

"And I remove things: `V14` added mask enhancements and `V15` dropped them, because they didn't earn their complexity."

Q What are you most proud of?

A "The economic architecture — that the expensive thing happens once and the thing users actually do is free. It's not the most complex code in the project, but it's the decision the entire business model rests on. If every colour change cost ₹5, there is no product at ₹999 a month."

"And technically, the scene-light anchoring in the shader — because it's a capability I got for free from a decision made two steps earlier in the pipeline. Repainting surfaces to a known white during cleanup was done to make the canvas look freshly painted; it turned out to also make the photo an illumination map. Noticing that connection was the most satisfying moment in the project."

Q What did you struggle with?

A "Scope. It's a solo project with no product manager, so there was nothing stopping me building the interesting thing instead of the necessary one. The billing rewrite is the honest evidence — I built a wallet system, shipped it, and then removed it. Better upfront thinking about what money model I actually needed would have saved weeks."

"The second is knowing when to stop hardening. I can always add another rate limit, another test, another guard. Deciding that the current security posture is *enough for this stage* and moving on is a judgement call I found genuinely hard."

Q How would you onboard another developer?

A "Most of it's already written, which was deliberate. There's a 17 KB README covering architecture, module map, endpoints, setup, and a production checklist. `AUTH_FLOW.md` is a request-by-request deep dive on authentication. `RAZORPAY_SETUP.md` takes you from creating an account to going live. `DEPLOYMENT.md` is the hardening checklist. Plus a Postman collection and a 10 KB annotated `.env.example`."

"Practically: `docker compose up` gives them Postgres, Redis and the backend, and `HUEVISTA_TESTING_STUB_AI_ENABLED=true` lets them click through the entire product without any AI credentials or spend. Then I'd have them fix a small bug in one feature module — because the packages are feature-sliced, a single module is a complete, comprehensible unit."

Q How do you keep code quality up working alone?

A "Automate the things a reviewer would catch: `./mvnw verify`, ESLint, `tsc --noEmit` and 479 tests on every PR, plus Dependabot. Then encode rules as tests rather than intentions — the migration-coverage test, the contract test, the middleware route-list test, the BFF allow-list test. Each one exists because it's a class of drift I know I'd miss by eye."

"And I write comments that explain *why*, not what. The codebase is full of them: why the webhook returns 5xx, why the rate limiter fails open, why `pdfImageLimit` doesn't scale with quantity, why dashboard isn't a grantable feature. Those are the decisions I'd otherwise re-litigate with myself in six months — and they're also what makes the codebase interview-ready, because I can point at the reasoning rather than reconstruct it."

35 · Traps, honest answers & what NOT to say

35.1 · Questions designed to catch you out

Question	What they're testing / how to answer
"So it's just API calls to Claude?"	Testing whether you'll get defensive. Agree partly, then redirect to the engineering *around* the calls: cost architecture, failure semantics, salvage rules, ΔE grounding, the reserve/commit ledger.
"Did you write this or did AI?"	Answer plainly. Whatever your process was, you must be able to explain **every decision** — which is what this document is for. The defence isn't a claim about authorship; it's fluency under questioning.
"How many real users?"	Don't inflate. "It's built and deployable but pre-launch / in pilot" is respectable. Follow with what you'd measure first: activation rate, projects per shop per week, segmentation success rate.
"Have you load-tested it?"	"No. The bounds are reasoned, not measured — pools, queues and semaphores are all capped, but I haven't run k6 to find the actual knee." Then say what you'd test first: the segmentation queue under burst.
"What's your test coverage percentage?"	Don't invent a number. "I haven't wired up JaCoCo — 479 tests across 53 backend classes and 40 frontend suites, weighted toward integration. Coverage percentage is on my CI to-do list, with a threshold gate."
"Why not use library X?"	Never say "I didn't know about it." Say what you'd gain and what it would cost, then whether that trade makes sense at this stage.
"This seems over-engineered for a solo project."	Agree where true. "Some of it is — the feature-grant system exists for one distributor's requirement. But the parts that look heavy are where money or security lives, and those are the places where under-engineering is expensive."
"What if Replicate shuts down tomorrow?"	"The mask pipeline is behind `ReplicateMaskSegmenter`, so swapping providers is one class. SAM 2 is open-weights and self-hostable. Existing projects are unaffected — recolouring is entirely client-side. The exposure is new-project creation, not the product."

35.2 · Phrases to avoid, and what to say instead

Don't say	Say instead
"It's fully production-ready and scales infinitely."	"It's deployable, with a documented hardening checklist. Here's what I'd fix before real traffic."
"I used X because it's the best."	"I used X because [constraint]. The trade-off was [cost]. I'd switch to Y if [trigger]."
"That's not really important."	"I deprioritised that because [reason] — it's on the list."
"I don't know."	"I haven't hit that. My instinct would be [reasoning] — how do you handle it?"
"The AI does it."	"The model produces X; I then do Y deterministically to make it reliable."
"I built everything myself."	"I made these decisions and can defend each one." (Focus on judgement, not keystrokes.)

35.3 · Five things to volunteer before they ask

1. **The Razorpay plan-amount check is missing.** Naming your own top risk reads as ownership.
2. **Timestamps are zone-naive** and the mitigation is a pinned JVM timezone, not a fix.
3. **No end-to-end tests.** Playwright across both repos is the biggest testing gap.
4. **No load testing.** The bounds are reasoned, not measured.
5. **CD isn't wired up.** CI runs on every PR; deploys are manual.

> **WHY VOLUNTEERING WEAKNESSES WORKS**
>
> An interviewer's job is to find the edges of your knowledge. If you name them first, you control the framing and you demonstrate the single most valuable trait a junior engineer can show: **knowing what you don't know.** A candidate who says "the plan-amount check is missing and here's the fix" is more trustworthy than one who has to be caught.

36 · Rapid-fire cheat sheet

If they ask…	Say
What is it?	B2B SaaS: paint shops show walk-ins their own room repainted in real catalogue shades before buying.
Stack in one line?	Spring Boot 4 / Java 17 / PostgreSQL / Redis / S3 backend; Next.js 16 / React 19 / TypeScript / WebGL2 frontend; Razorpay; Claude + Replicate.
The one insight?	AI once per project for the mask; WebGL forever for the colour. ~₹5 → ₹0, 30 s → 60 fps.
Scale of the code?	78K LOC, 207 endpoints, 43 entities, 479 tests, 37 migrations.
Why not generative AI?	₹3–8 per render, 10–30 s latency, hallucinates the customer's furniture.
Auth in one line?	HS256 JWT + SHA-256-hashed refresh tokens, in HttpOnly cookies, proxied by a Next BFF — the browser never holds a token.
Razorpay in one line?	Subscriptions + Orders, HMAC-verified at Checkout and on webhooks, idempotent on event ID, 5xx to trigger retries.
Idempotency?	`ProcessedWebhookEvent` keyed on `X-Razorpay-Event-Id`, inserted inside the handler's transaction; PK collision is the concurrency control.
Fail open or closed?	Rate limiter: **open** (availability). Webhook signature: **closed** (integrity). Same question, opposite answers.
Quota safety?	Reserve → commit / refund. Closes the TOCTOU race and refunds on pipeline failure.
Queue?	Redis RPOPLPUSH reliable queue; ack on completion; requeue stale after 10 min; drop after 3 attempts; depth cap 500.
Thread starvation fix?	`Semaphore(8)` bulkhead on synchronous SAM 2 calls; 503 beyond it.
Shader in one line?	Split photo into blurred *form* and high-frequency *detail*; paint = target × form + detail.
Colour matching?	sRGB → linear → XYZ → CIELAB, CIE76 ΔE. RGB distance ≠ perceived distance.
Schema management?	Flyway owns it (37 migrations); Hibernate runs `ddl-auto=validate` and never mutates.
Money type?	`int` paise. Never floating point.
Biggest risk?	No check that the Razorpay dashboard plan amount matches `Plan.java`.
Biggest gap?	No end-to-end tests, no load testing, CD not wired up.
What breaks at 100×?	Segmentation — on cost and Replicate limits before my own capacity.
What would you change?	`Instant` / `timestamptz` instead of `LocalDateTime`; Flyway from day one; OpenAPI-generated types.

HueVista — Project & Interview Handbook

Generated from the source of `VikramMali14/HueVista` and `VikramMali14/HueVistaFrontEnd`.
Every figure, file path and code excerpt in this document was read from the repositories.